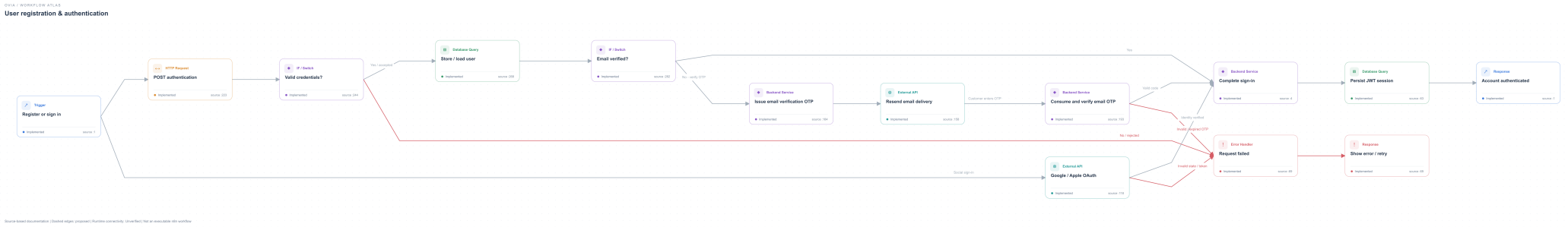
User registration & authentication

Architecture overview. Readable node and connection sheets follow.



User registration & authentication

Readable detail sheet 1/4. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Register or sign in

Account authentication forms and verification experience.

src/components/auth-experience.tsx:1

Next -> #2 (p2); Social sign-in -> #14 (p5)

#02 / HTTP Request / Implemented

POST authentication

Registration and login share the dispatcher. Password, email and terms requirements differ.

src/app/api/v1/[[...path]]/route.ts:223

Next -> #3 (p2)

#03 / IF / Switch / Implemented

Valid credentials?

Zod validation, password policy, duplicate email checks and scrypt password verification.

src/app/api/v1/[[...path]]/route.ts:244

Yes / accepted -> #4 (p2); No / rejected -> #11 (p4)

#04 / Database Query / Implemented

Store / load user

users: id, email, passwordHash, verification and terms fields.

src/app/api/v1/[[...path]]/route.ts:268

Next -> #5 (p3)

User registration & authentication

Readable detail sheet 2/4. Node numbers and page references preserve every directed connection.

#05 / IF / Switch / Implemented

Email verified?

Unverified accounts receive an email OTP. Verified accounts proceed to completeSignIn.

src/app/api/v1/[[...path]]/route.ts:282

Yes -> #8 (p3); No - verify OTP -> #6 (p3)

#06 / Backend Service / Implemented

Issue email verification OTP

Generate and hash a one-time verification code with expiration and attempt limits.

src/server/recovery.ts:184

Next -> #7 (p3)

#07 / External API / Implemented

Resend email delivery

Email provider HTTP call; failures cannot be treated as successful delivery. Runtime configuration and connectivity are Unverified.

src/server/mail.ts:158

Customer enters OTP -> #13 (p5)

#08 / Backend Service / Implemented

Complete sign-in

Merge guest cart and account state, then create the session.

src/server/account-sign-in.ts:4

Next -> #9 (p4)

User registration & authentication

Readable detail sheet 3/4. Node numbers and page references preserve every directed connection.

#09 / Database Query / Implemented

Persist JWT session

HS256 JWT in HttpOnly cookie; hashed jti in sessions with userId and expiration.

src/server/auth.ts:63

Next -> #10 (p4)

#10 / Response / Implemented

Account authenticated

Successful sign-in returns safe user fields and navigates the customer account.

src/components/auth-experience.tsx:1

END: no outgoing connection

#11 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #12 (p4)

#12 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

User registration & authentication

Readable detail sheet 4/4. Node numbers and page references preserve every directed connection.

#13 / Backend Service / Implemented

Consume and verify email OTP

Submitted code is compared with the hashed token, checked for expiry and attempt limits, then consumed.

src/server/recovery.ts:193

Valid code -> #8 (p3); Invalid / expired OTP -> #11 (p4)

#14 / External API / Implemented

Google / Apple OAuth

State and PKCE flow, code exchange, JWKS identity verification and resolveSocialUser. Runtime configuration and connectivity are Unverified.

src/server/social-auth.ts:118

Identity verified -> #8 (p3); Invalid state / token -> #11 (p4)

User registration & authentication

Complete directed connection register / no omitted success, failure or proposed edges

#01 Register or sign in -> #02 POST authentication | Next

#02 POST authentication -> #03 Valid credentials? | Next

#03 Valid credentials? -> #04 Store / load user | Yes / accepted

#04 Store / load user -> #05 Email verified? | Next

#06 Issue email verification OTP -> #07 Resend email delivery | Next

#08 Complete sign-in -> #09 Persist JWT session | Next

#09 Persist JWT session -> #10 Account authenticated | Next

#03 Valid credentials? -> #11 Request failed | No / rejected | FAILURE

#11 Request failed -> #12 Show error / retry | Next | FAILURE

#05 Email verified? -> #08 Complete sign-in | Yes

#05 Email verified? -> #06 Issue email verification OTP | No - verify OTP

#07 Resend email delivery -> #13 Consume and verify email OTP | Customer enters OTP

#13 Consume and verify email OTP -> #08 Complete sign-in | Valid code

#13 Consume and verify email OTP -> #11 Request failed | Invalid / expired OTP | FAILURE

#01 Register or sign in -> #14 Google / Apple OAuth | Social sign-in

#14 Google / Apple OAuth -> #08 Complete sign-in | Identity verified

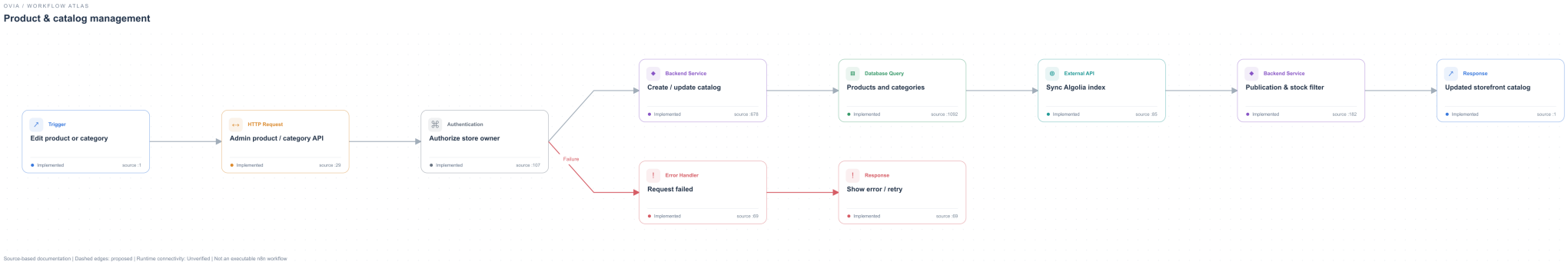
#14 Google / Apple OAuth -> #11 Request failed | Invalid state / token | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: The OTP delivery and verification nodes apply only to unverified email accounts. The Yes branch below bypasses OTP. Google and Apple use a separate OAuth branch.

Product & catalog management

Architecture overview. Readable node and connection sheets follow.



Product & catalog management

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Edit product or category

Owner catalog editor; create, update or remove catalog records.

src/components/admin/product-editor.tsx:1

Next -> #2 (p2)

#02 / HTTP Request / Implemented

Admin product / category API

Category CRUD has its own routes; inspect the API inventory for each method.

src/app/api/admin/products/route.ts:29

Next -> #3 (p2)

#03 / Authentication / Implemented

Authorize store owner

JWT session plus normalized OWNER or ADMIN role.

src/server/admin-auth.ts:107

Next -> #4 (p2); Failure -> #9 (p4)

#04 / Backend Service / Implemented

Create / update catalog

Server normalizes product data, variants and media. Category records are maintained separately.

src/server/admin-service.ts:678

Next -> #5 (p3)

Product & catalog management

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

Products and categories

Persist actual catalog collections; category matching includes name, slug and department logic rather than a declared foreign key.

src/server/admin-service.ts:1092

Next -> #6 (p3)

#06 / External API / Implemented

Sync Algolia index

Writes run only when a write key exists. Index errors are logged and are non-blocking. Runtime configuration and connectivity are Unverified.

src/lib/algolia.ts:85

Next -> #7 (p3)

#07 / Backend Service / Implemented

Publication & stock filter

Storefront excludes unpublished, draft, archived and imported/demo records and overlays inventory balances.

src/server/db.ts:182

Next -> #8 (p3)

#08 / Response / Implemented

Updated storefront catalog

Catalog listings display published products and supported facets.

src/components/catalog-view.tsx:1

END: no outgoing connection

Product & catalog management

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #10 (p4)

#10 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Product & catalog management

Complete directed connection register / no omitted success, failure or proposed edges

#01 Edit product or category -> #02 Admin product / category API | Next

#02 Admin product / category API -> #03 Authorize store owner | Next

#03 Authorize store owner -> #04 Create / update catalog | Next

#04 Create / update catalog -> #05 Products and categories | Next

#05 Products and categories -> #06 Sync Algolia index | Next

#06 Sync Algolia index -> #07 Publication & stock filter | Next

#07 Publication & stock filter -> #08 Updated storefront catalog | Next

#03 Authorize store owner -> #09 Request failed | Failure | FAILURE

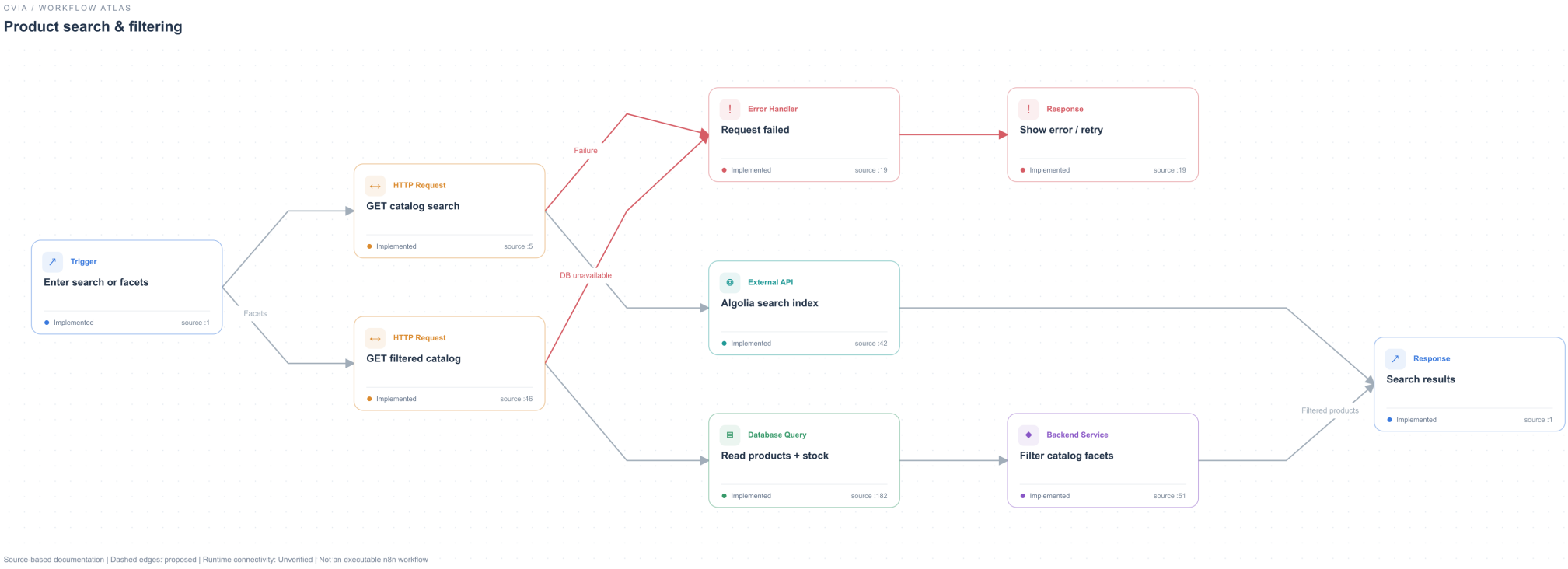
#09 Request failed -> #10 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: Algolia synchronization is optional and best effort; successful database writes do not prove the search index was updated.

Product search & filtering

Architecture overview. Readable node and connection sheets follow.



Product search & filtering

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Enter search or facets

Instant search modal and catalog filter UI.

src/components/instant-search-modal.tsx:1

Next -> #2 (p2); Facets -> #7 (p3)

#02 / HTTP Request / Implemented

GET catalog search

Query, limit and page are normalized; the route enforces a ten-second search timeout.

src/app/api/search/route.ts:5

Next -> #3 (p2); Failure -> #5 (p3)

#03 / External API / Implemented

Algolia search index

searchSingleIndex queries the configured products index. Runtime configuration and connectivity are Unverified.

src/lib/algolia.ts:42

Next -> #4 (p2)

#04 / Response / Implemented

Search results

Render returned hits and links to products.

src/components/instant-search-modal.tsx:1

END: no outgoing connection

Product search & filtering

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Error Handler / Implemented

Request failed

Unavailable configuration, timeout or provider failure returns HTTP 503.

src/app/api/search/route.ts:19

Next -> #6 (p3)

#06 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/app/api/search/route.ts:19

END: no outgoing connection

#07 / HTTP Request / Implemented

GET filtered catalog

Database catalog filtering via filterProducts.

src/app/api/v1/[[...path]]/route.ts:46

Next -> #9 (p4); DB unavailable -> #5 (p3)

#08 / Backend Service / Implemented

Filter catalog facets

Category, department and other Filters constrain product results.

src/lib/catalog-filter.ts:51

Filtered products -> #4 (p2)

Product search & filtering

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Database Query / Implemented

Read products + stock

Database catalog and SKU inventory balances.

src/server/db.ts:182

Next -> #8 (p3)

Product search & filtering

Complete directed connection register / no omitted success, failure or proposed edges

#01 Enter search or facets -> #02 GET catalog search | Next

#02 GET catalog search -> #03 Algolia search index | Next

#03 Algolia search index -> #04 Search results | Next

#02 GET catalog search -> #05 Request failed | Failure | FAILURE

#05 Request failed -> #06 Show error / retry | Next | FAILURE

#01 Enter search or facets -> #07 GET filtered catalog | Facets

#07 GET filtered catalog -> #09 Read products + stock | Next

#09 Read products + stock -> #08 Filter catalog facets | Next

#08 Filter catalog facets -> #04 Search results | Filtered products

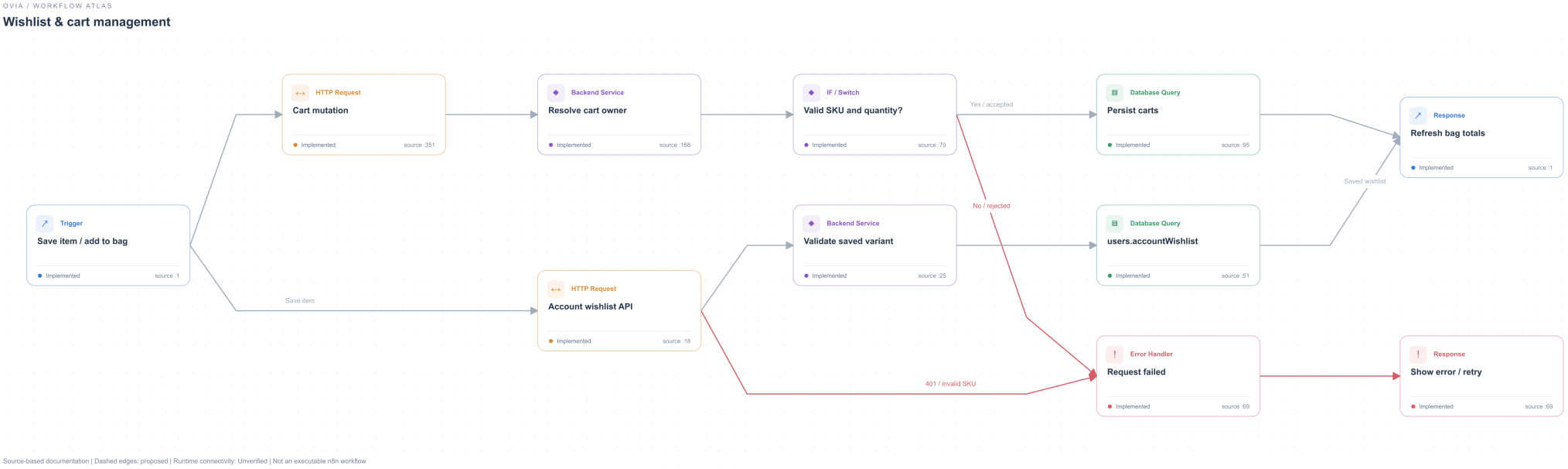
#07 GET filtered catalog -> #05 Request failed | DB unavailable | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: GET /api/v1/products and /api/v1/search filter the MongoDB catalog; /api/search uses Algolia.

Wishlist & cart management

Architecture overview. Readable node and connection sheets follow.



Wishlist & cart management

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Save item / add to bag

Store provider coordinates catalog, wishlist and cart interactions.

src/components/store-provider.tsx:1

Next -> #2 (p2); Save item -> #9 (p4)

#02 / HTTP Request / Implemented

Cart mutation

PATCH and DELETE /api/v1/cart/items/:sku are also implemented.

src/app/api/v1/[[...path]]/route.ts:351

Next -> #3 (p2)

#03 / Backend Service / Implemented

Resolve cart owner

Signed-in user ID or SHA256-hashed guest cookie identifies the cart.

src/server/auth.ts:168

Next -> #4 (p2)

#04 / IF / Switch / Implemented

Valid SKU and quantity?

Available variant, quantity 0-10 and optional cart version check; stale versions return 409.

src/server/commerce.ts:70

Yes / accepted -> #5 (p3); No / rejected -> #7 (p3)

Wishlist & cart management

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

Persist carts

Cart contains id, lines[].sku, lines[].quantity and version.

src/server/commerce.ts:95

Next -> #6 (p3)

#06 / Response / Implemented

Refresh bag totals

Render server-confirmed line items and totals.

src/components/shopping-bag-drawer.tsx:1

END: no outgoing connection

#07 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #8 (p3)

#08 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Wishlist & cart management

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / HTTP Request / Implemented

Account wishlist API

GET reads and DELETE removes saved items; authentication required.

src/app/api/v1/account/wishlist/route.ts:18

Next -> #10 (p4); 401 / invalid SKU -> #7 (p3)

#10 / Backend Service / Implemented

Validate saved variant

Validate input and catalog SKU.

src/server/account-wishlist.ts:25

Next -> #11 (p4)

#11 / Database Query / Implemented

users.accountWishlist

Wishlist is embedded on users; no standalone wishlist collection is invented.

src/server/account-wishlist.ts:51

Saved wishlist -> #6 (p3)

Wishlist & cart management

Complete directed connection register / no omitted success, failure or proposed edges

#01 Save item / add to bag -> #02 Cart mutation | Next

#02 Cart mutation -> #03 Resolve cart owner | Next

#03 Resolve cart owner -> #04 Valid SKU and quantity? | Next

#04 Valid SKU and quantity? -> #05 Persist carts | Yes / accepted

#05 Persist carts -> #06 Refresh bag totals | Next

#04 Valid SKU and quantity? -> #07 Request failed | No / rejected | FAILURE

#07 Request failed -> #08 Show error / retry | Next | FAILURE

#01 Save item / add to bag -> #09 Account wishlist API | Save item

#09 Account wishlist API -> #10 Validate saved variant | Next

#10 Validate saved variant -> #11 users.accountWishlist | Next

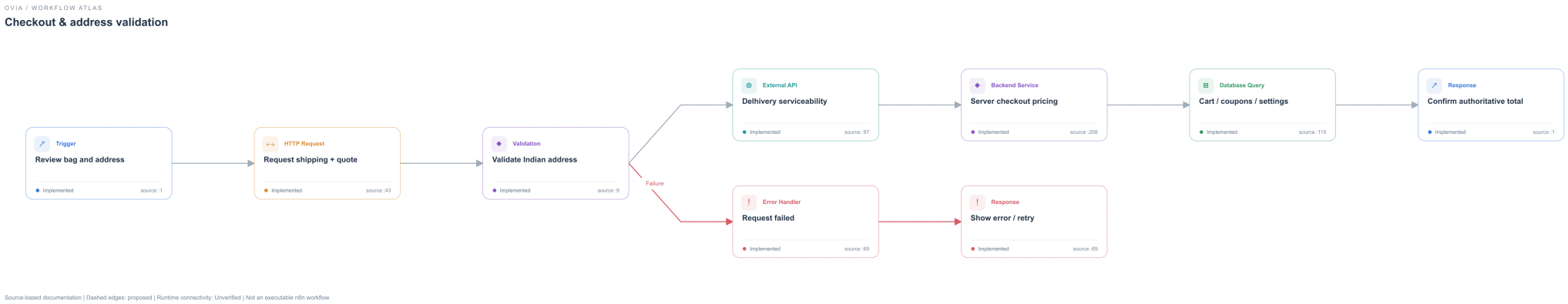
#11 users.accountWishlist -> #06 Refresh bag totals | Saved wishlist

#09 Account wishlist API -> #07 Request failed | 401 / invalid SKU | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Checkout & address validation

Architecture overview. Readable node and connection sheets follow.



Checkout & address validation

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Review bag and address

Checkout page collects customer address and shipping preferences.

src/app/checkout/page.tsx:1

Next -> #2 (p2)

#02 / HTTP Request / Implemented

Request shipping + quote

Quote input is strict; origin and rate-limit checks apply.

src/app/api/v1/checkout/[[...action]]/route.ts:43

Next -> #3 (p2)

#03 / Validation / Implemented

Validate Indian address

Country IN, six-digit PIN, email, city and region. Checkout input requires a valid Indian mobile number.

src/server/commerce.ts:9

Next -> #4 (p2); Failure -> #8 (p3)

#04 / External API / Implemented

Delhivery serviceability

Checkout page separately calls shipping/serviceability for the PIN code. Runtime configuration and connectivity are Unverified.

src/lib/shipping/delhivery.ts:97

Next -> #5 (p3)

Checkout & address validation

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Backend Service / Implemented

Server checkout pricing

Price catalog lines, shipping and coupon rules using server records.

src/server/checkout-pricing.ts:208

Next -> #6 (p3)

#06 / Database Query / Implemented

Cart / coupons / settings

Persisted cart version and current discount data are used to construct the quote.

src/server/checkout-pricing.ts:115

Next -> #7 (p3)

#07 / Response / Implemented

Confirm authoritative total

Frontend displays quote and passes quoteId to secure checkout.

src/app/checkout/page.tsx:1

END: no outgoing connection

#08 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #9 (p4)

Checkout & address validation

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Checkout & address validation

Complete directed connection register / no omitted success, failure or proposed edges

#01 Review bag and address -> #02 Request shipping + quote | Next

#02 Request shipping + quote -> #03 Validate Indian address | Next

#03 Validate Indian address -> #04 Delhivery serviceability | Next

#04 Delhivery serviceability -> #05 Server checkout pricing | Next

#05 Server checkout pricing -> #06 Cart / coupons / settings | Next

#06 Cart / coupons / settings -> #07 Confirm authoritative total | Next

#03 Validate Indian address -> #08 Request failed | Failure | FAILURE

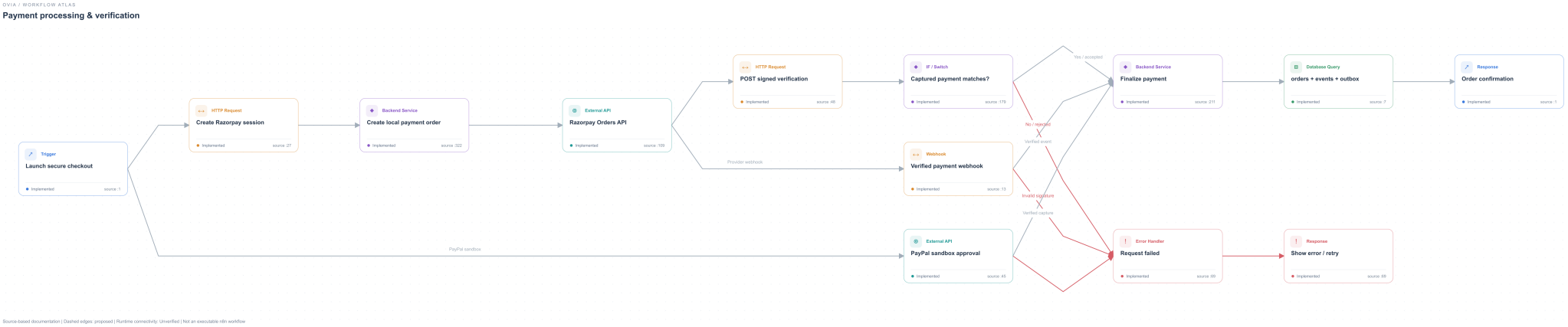
#08 Request failed -> #09 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: Serviceability is a separate frontend request, not a sequential call inside checkoutQuote. The diagram expresses the checkout interaction, not a single handler stack.

Payment processing & verification

Architecture overview. Readable node and connection sheets follow.



Payment processing & verification

Readable detail sheet 1/4. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Launch secure checkout

Customer confirms the reviewed quote and starts provider checkout.

src/app/checkout/page.tsx:1

Next -> #2 (p2); PayPal sandbox -> #13 (p5)

#02 / HTTP Request / Implemented

Create Razorpay session

Validate input, owner, origin and rate limits before beginRazorpayCheckout.

src/app/api/v1/payments/razorpay/[[...action]]/route.ts:27

Next -> #3 (p2)

#03 / Backend Service / Implemented

Create local payment order

createOrder reserves stock and creates a durable order; provider order data is attached.

src/server/razorpay-checkout.ts:322

Next -> #4 (p2)

#04 / External API / Implemented

Razorpay Orders API

Authenticated provider requests create and retrieve payment orders. Runtime configuration and connectivity are Unverified.

src/server/razorpay-checkout.ts:109

Next -> #5 (p3); Provider webhook -> #12 (p4)

Payment processing & verification

Readable detail sheet 2/4. Node numbers and page references preserve every directed connection.

#05 / HTTP Request / Implemented

POST signed verification

verifyRazorpayPayment validates returned payment identifiers and signature.

src/app/api/v1/payments/razorpay/[[...action]]/route.ts:48

Next -> #6 (p3)

#06 / IF / Switch / Implemented

Captured payment matches?

Amount, currency, remote order identity and capture status must match the local order.

src/server/razorpay-checkout.ts:179

Yes / accepted -> #7 (p3); No / rejected -> #10 (p4)

#07 / Backend Service / Implemented

Finalize payment

Duplicate events are checked; timely captures become paid, late captures require refund review.

src/server/commerce.ts:211

Next -> #8 (p3)

#08 / Database Query / Implemented

orders + events + outbox

Payment metadata remains on orders; event IDs protect replay; confirmation mail is queued only for confirmed purchases.

src/server/commerce.ts:7

Next -> #9 (p4)

Payment processing & verification

Readable detail sheet 3/4. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Order confirmation

Customer receives verified order state, never an assumed successful capture.

src/app/orders/[id]/confirmation/page.tsx:1

END: no outgoing connection

#10 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #11 (p4)

#11 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

#12 / Webhook / Implemented

Verified payment webhook

Stripe, Razorpay and PayPal dispatch; signature verification precedes reconciliation.

src/app/api/webhooks/[provider]/route.ts:13

Verified event -> #7 (p3); Invalid signature -> #10 (p4)

Payment processing & verification

Readable detail sheet 4/4. Node numbers and page references preserve every directed connection.

#13 / External API / Implemented

PayPal sandbox approval

Create sandbox order, redirect approval, capture and verify provider response. Runtime configuration and connectivity are Unverified.

src/server/paypal-checkout.ts:45

Verified capture -> #7 (p3); Next -> #10 (p4)

Payment processing & verification

Complete directed connection register / no omitted success, failure or proposed edges

#01 Launch secure checkout -> #02 Create Razorpay session | Next

#02 Create Razorpay session -> #03 Create local payment order | Next

#03 Create local payment order -> #04 Razorpay Orders API | Next

#04 Razorpay Orders API -> #05 POST signed verification | Next

#05 POST signed verification -> #06 Captured payment matches? | Next

#06 Captured payment matches? -> #07 Finalize payment | Yes / accepted

#07 Finalize payment -> #08 orders + events + outbox | Next

#08 orders + events + outbox -> #09 Order confirmation | Next

#06 Captured payment matches? -> #10 Request failed | No / rejected | FAILURE

#10 Request failed -> #11 Show error / retry | Next | FAILURE

#04 Razorpay Orders API -> #12 Verified payment webhook | Provider webhook

#12 Verified payment webhook -> #07 Finalize payment | Verified event

#12 Verified payment webhook -> #10 Request failed | Invalid signature | FAILURE

#01 Launch secure checkout -> #13 PayPal sandbox approval | PayPal sandbox

#13 PayPal sandbox approval -> #07 Finalize payment | Verified capture

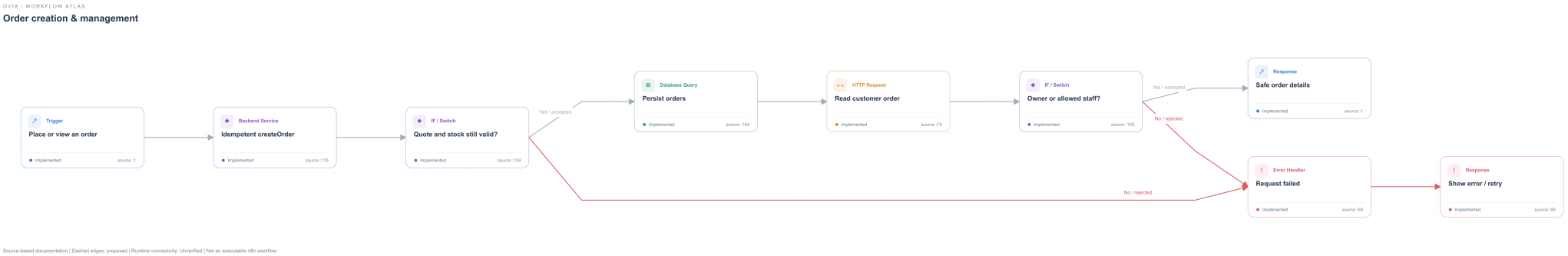
#13 PayPal sandbox approval -> #10 Request failed | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: PayPal uses a separate implemented sandbox create/approve/capture flow. Legacy Stripe checkout is guarded to test credentials.

Order creation & management

Architecture overview. Readable node and connection sheets follow.



Order creation & management

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Place or view an order

Customer order detail and post-checkout management.

src/components/account-order-details.tsx:1

Next -> #2 (p2)

#02 / Backend Service / Implemented

Idempotent createOrder

Existing owner + idempotencyKey returns the prior compatible order; active checkout is reused.

src/server/commerce.ts:115

Next -> #3 (p2)

#03 / IF / Switch / Implemented

Quote and stock still valid?

Empty carts, changed quoteId, incompatible providers and insufficient stock are rejected.

src/server/commerce.ts:154

Yes / accepted -> #4 (p2); No / rejected -> #8 (p3)

#04 / Database Query / Implemented

Persist orders

Order contains address, line snapshots, provider IDs, owner, userId and expiry.

src/server/commerce.ts:184

Next -> #5 (p3)

Order creation & management

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / HTTP Request / Implemented

Read customer order

Lookup by ID or order number; ownership and allowed staff roles are checked.

src/app/api/v1/[[...path]]/route.ts:79

Next -> #6 (p3)

#06 / IF / Switch / Implemented

Owner or allowed staff?

Unauthorized access returns 403; owner and idempotencyKey fields are removed from public responses.

src/app/api/v1/[[...path]]/route.ts:105

Yes / accepted -> #7 (p3); No / rejected -> #8 (p3)

#07 / Response / Implemented

Safe order details

Render payment state and courier-verified shipping data.

src/components/account-order-details.tsx:1

END: no outgoing connection

#08 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #9 (p4)

Order creation & management

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Order creation & management

Complete directed connection register / no omitted success, failure or proposed edges

#01 Place or view an order -> #02 Idempotent createOrder | Next

#02 Idempotent createOrder -> #03 Quote and stock still valid? | Next

#03 Quote and stock still valid? -> #04 Persist orders | Yes / accepted

#04 Persist orders -> #05 Read customer order | Next

#05 Read customer order -> #06 Owner or allowed staff? | Next

#06 Owner or allowed staff? -> #07 Safe order details | Yes / accepted

#03 Quote and stock still valid? -> #08 Request failed | No / rejected | FAILURE

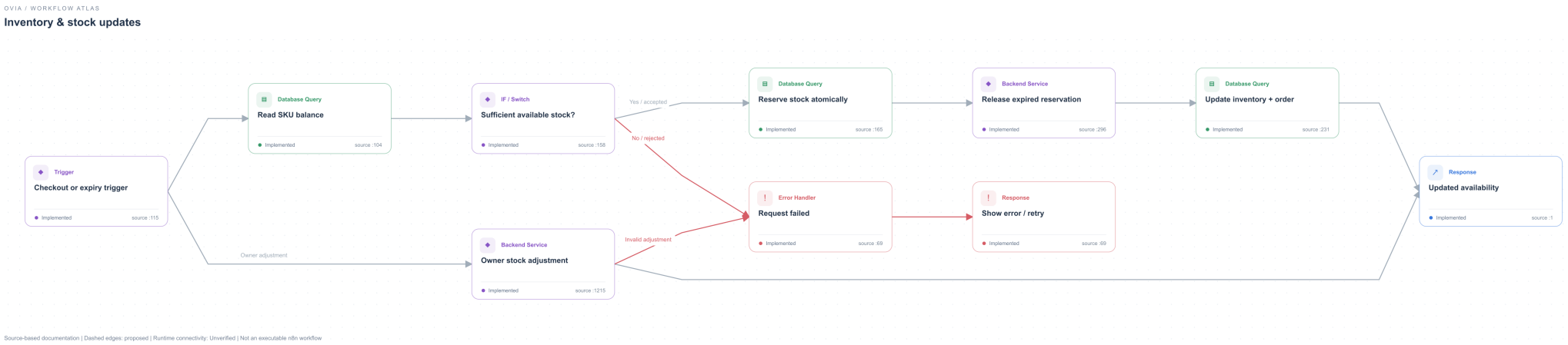
#06 Owner or allowed staff? -> #08 Request failed | No / rejected | FAILURE

#08 Request failed -> #09 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Inventory & stock updates

Architecture overview. Readable node and connection sheets follow.



Inventory & stock updates

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Checkout or expiry trigger

Reservation runs within checkout order creation; expiry runs via protected cron.

src/server/commerce.ts:115

Next -> #2 (p2); Owner adjustment -> #10 (p4)

#02 / Database Query / Implemented

Read SKU balance

inventory.id equals variant SKU; available quantity is loaded or derived from product variants.

src/server/commerce.ts:104

Next -> #3 (p2)

#03 / IF / Switch / Implemented

Sufficient available stock?

Reject overselling with HTTP 409 before updating the order.

src/server/commerce.ts:158

Yes / accepted -> #4 (p2); No / rejected -> #8 (p3)

#04 / Database Query / Implemented

Reserve stock atomically

Balance decrement and order insert share the MongoDB transaction.

src/server/commerce.ts:165

Next -> #5 (p3)

Inventory & stock updates

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Backend Service / Implemented

Release expired reservation

Expiry code releases nonfinal inventory once; cancellation has its own release path.

src/server/commerce.ts:296

Next -> #6 (p3)

#06 / Database Query / Implemented

Update inventory + order

inventoryReleased protects repeated release.

src/server/commerce.ts:231

Next -> #7 (p3)

#07 / Response / Implemented

Updated availability

Catalog overlays inventory on variant stock.

src/components/product-detail.tsx:1

END: no outgoing connection

#08 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #9 (p4)

Inventory & stock updates

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

#10 / Backend Service / Implemented

Owner stock adjustment

PATCH /api/admin/inventory updates inventory, product variant stock, inventoryMovements and audit.

src/server/admin-service.ts:1215

Next -> #7 (p3); Invalid adjustment -> #8 (p3)

Inventory & stock updates

Complete directed connection register / no omitted success, failure or proposed edges

#01 Checkout or expiry trigger -> #02 Read SKU balance | Next

#02 Read SKU balance -> #03 Sufficient available stock? | Next

#03 Sufficient available stock? -> #04 Reserve stock atomically | Yes / accepted

#04 Reserve stock atomically -> #05 Release expired reservation | Next

#05 Release expired reservation -> #06 Update inventory + order | Next

#06 Update inventory + order -> #07 Updated availability | Next

#03 Sufficient available stock? -> #08 Request failed | No / rejected | FAILURE

#08 Request failed -> #09 Show error / retry | Next | FAILURE

#01 Checkout or expiry trigger -> #10 Owner stock adjustment | Owner adjustment

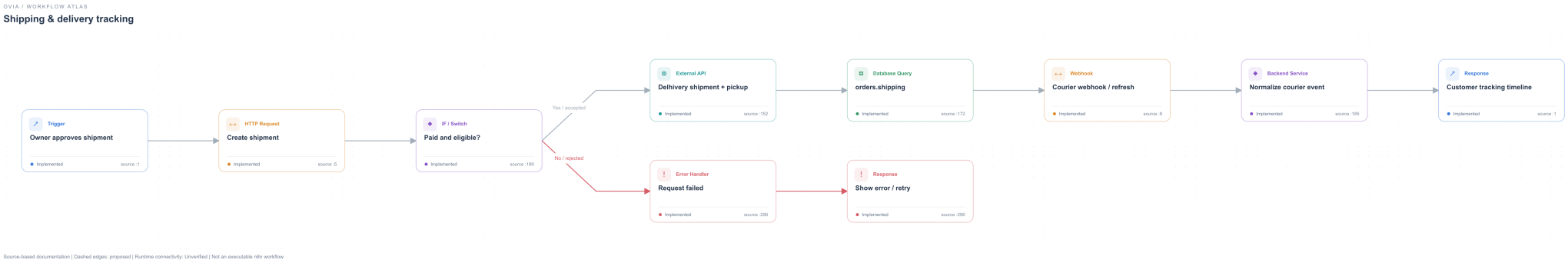
#10 Owner stock adjustment -> #07 Updated availability | Next

#10 Owner stock adjustment -> #08 Request failed | Invalid adjustment | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Shipping & delivery tracking

Architecture overview. Readable node and connection sheets follow.



Shipping & delivery tracking

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Owner approves shipment

Owner reviews order and initiates shipment / pickup.

src/app/admin/(control-center)/orders/[id]/page.tsx:1

Next -> #2 (p2)

#02 / HTTP Request / Implemented

Create shipment

Authorized owner executes controlled logistics.

src/app/api/admin/orders/[id]/shipment/route.ts:5

Next -> #3 (p2)

#03 / IF / Switch / Implemented

Paid and eligible?

Payment capture, unrefunded state, package data and logistics lease checks prevent unsafe repeated actions.

src/server/logistics.ts:189

Yes / accepted -> #4 (p2); No / rejected -> #9 (p4)

#04 / External API / Implemented

Delhivery shipment + pickup

Courier API returns AWB and shipment metadata; pickup is requested through the shipping service. Runtime configuration and connectivity are Unverified.

src/lib/shipping/delhivery.ts:152

Next -> #5 (p3)

Shipping & delivery tracking

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

orders.shipping

Shipping data and logistics operation progress are embedded on the order.

src/server/logistics.ts:172

Next -> #6 (p3)

#06 / Webhook / Implemented

Courier webhook / refresh

Courier events are authenticated; refresh route can also reconcile tracking.

src/app/api/webhooks/delhivery/route.ts:8

Next -> #7 (p3)

#07 / Backend Service / Implemented

Normalize courier event

Match AWB to stored order and map courier status into internal shipping status.

src/lib/shipping/service.ts:185

Next -> #8 (p3)

#08 / Response / Implemented

Customer tracking timeline

Only verified shipment information is shown to customers.

src/components/order-tracking.tsx:1

END: no outgoing connection

Shipping & delivery tracking

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Error Handler / Implemented

Request failed

Ambiguous courier writes are tracked for review instead of falsely declaring shipment success.

src/server/logistics.ts:296

Next -> #10 (p4)

#10 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/logistics.ts:296

END: no outgoing connection

Shipping & delivery tracking

Complete directed connection register / no omitted success, failure or proposed edges

#01 Owner approves shipment -> #02 Create shipment | Next

#02 Create shipment -> #03 Paid and eligible? | Next

#03 Paid and eligible? -> #04 Delhivery shipment + pickup | Yes / accepted

#04 Delhivery shipment + pickup -> #05 orders.shipping | Next

#05 orders.shipping -> #06 Courier webhook / refresh | Next

#06 Courier webhook / refresh -> #07 Normalize courier event | Next

#07 Normalize courier event -> #08 Customer tracking timeline | Next

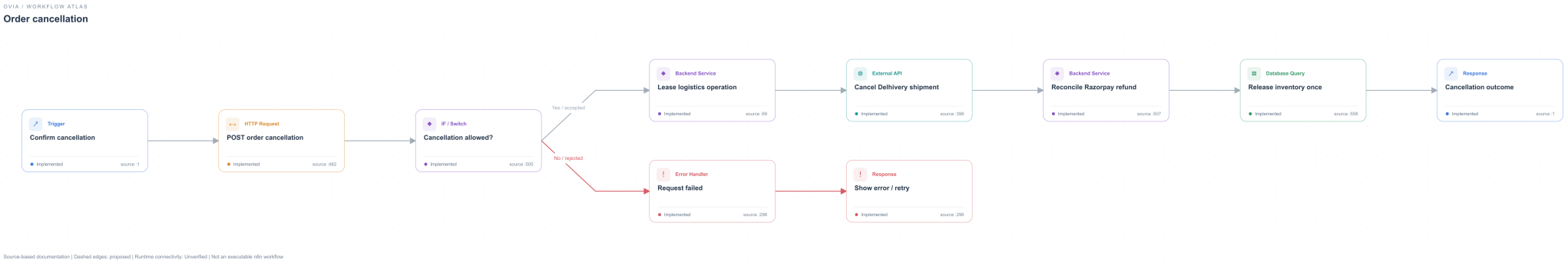
#03 Paid and eligible? -> #09 Request failed | No / rejected | FAILURE

#09 Request failed -> #10 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Order cancellation

Architecture overview. Readable node and connection sheets follow.



Order cancellation

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Confirm cancellation

Customer confirms cancellation and optional reason.

src/components/account-order-details.tsx:1

Next -> #2 (p2)

#02 / HTTP Request / Implemented

POST order cancellation

Validate owner, deadline and confirmed=true payload.

src/app/api/v1/[[...path]]/route.ts:482

Next -> #3 (p2)

#03 / IF / Switch / Implemented

Cancellation allowed?

Expired deadline rejects the request; logistics checks prevent unsafe cancellation.

src/app/api/v1/[[...path]]/route.ts:505

Yes / accepted -> #4 (p2); No / rejected -> #9 (p4)

#04 / Backend Service / Implemented

Lease logistics operation

Tracks processing, failed or confirmed phases on the order and audit entries.

src/server/logistics.ts:69

Next -> #5 (p3)

Order cancellation

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / External API / Implemented

Cancel Delhivery shipment

Cancel a shipment when present; courier confirmation is required before final reconciliation. Runtime configuration and connectivity are Unverified.

src/lib/shipping/delhivery.ts:396

Next -> #6 (p3)

#06 / Backend Service / Implemented

Reconcile Razorpay refund

Captured Razorpay payment is reconciled through the refund service. Other providers require appropriate review.

src/server/logistics.ts:507

Next -> #7 (p3)

#07 / Database Query / Implemented

Release inventory once

Update order cancellation, stock and audit; errors remain visible on logisticsOperations.cancel.

src/server/logistics.ts:558

Next -> #8 (p3)

#08 / Response / Implemented

Cancellation outcome

Customer sees confirmed or unresolved cancellation and refund status.

src/components/account-order-details.tsx:1

END: no outgoing connection

Order cancellation

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/logistics.ts:296

Next -> #10 (p4)

#10 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/logistics.ts:296

END: no outgoing connection

Order cancellation

Complete directed connection register / no omitted success, failure or proposed edges

#01 Confirm cancellation -> #02 POST order cancellation | Next

#02 POST order cancellation -> #03 Cancellation allowed? | Next

#03 Cancellation allowed? -> #04 Lease logistics operation | Yes / accepted

#04 Lease logistics operation -> #05 Cancel Delhivery shipment | Next

#05 Cancel Delhivery shipment -> #06 Reconcile Razorpay refund | Next

#06 Reconcile Razorpay refund -> #07 Release inventory once | Next

#07 Release inventory once -> #08 Cancellation outcome | Next

#03 Cancellation allowed? -> #09 Request failed | No / rejected | FAILURE

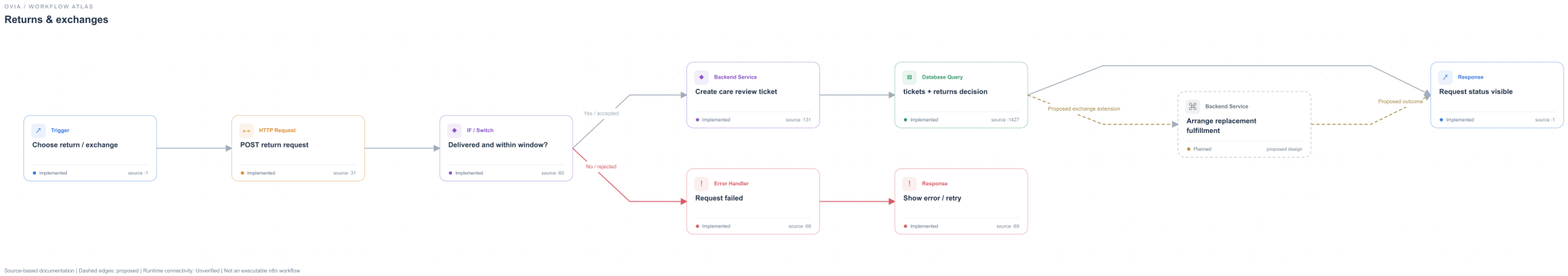
#09 Request failed -> #10 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: A cancelled order does not itself prove the payment provider has completed a refund.

Returns & exchanges

Architecture overview. Readable node and connection sheets follow.



Returns & exchanges

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Choose return / exchange

Return and exchange forms collect reason, pickup details, items and optional replacement SKU.

src/components/product-exchange-form.tsx:1

Next -> #2 (p2)

#02 / HTTP Request / Implemented

POST return request

Strict input schema and customerOrder ownership check.

src/app/api/v1/orders/[id]/return/route.ts:31

Next -> #3 (p2)

#03 / IF / Switch / Implemented

Delivered and within window?

Paid courier-confirmed delivery, no active refund and unexpired deadline are required. Exchange validates replacement availability without reserving it.

src/app/api/v1/orders/[id]/return/route.ts:60

Yes / accepted -> #4 (p2); No / rejected -> #7 (p3)

#04 / Backend Service / Implemented

Create care review ticket

Idempotent tickets and support conversation capture the requested order action.

src/server/care-order-requests.ts:131

Next -> #5 (p3)

Returns & exchanges

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

tickets + returns decision

Admin approves or rejects the request; returns stores inspection decision and links ticketId/orderRawId.

src/server/admin-service.ts:1427

Next -> #6 (p3); Proposed exchange extension -> #9 (p4) [proposed]

#06 / Response / Implemented

Request status visible

Customer can follow the support request; approval does not disburse funds.

src/components/account-orders-returns.tsx:1

END: no outgoing connection

#07 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #8 (p3)

#08 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Returns & exchanges

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Backend Service / Planned

Arrange replacement fulfillment

Proposed extension: reserve replacement stock, arrange reverse pickup and issue a replacement order after inspection. No implementation mapping is claimed.

Planned design; no source implementation claimed.

Proposed outcome -> #6 (p3) [proposed]

Returns & exchanges

Complete directed connection register / no omitted success, failure or proposed edges

#01 Choose return / exchange -> #02 POST return request | Next

#02 POST return request -> #03 Delivered and within window? | Next

#03 Delivered and within window? -> #04 Create care review ticket | Yes / accepted

#04 Create care review ticket -> #05 tickets + returns decision | Next

#05 tickets + returns decision -> #06 Request status visible | Next

#03 Delivered and within window? -> #07 Request failed | No / rejected | FAILURE

#07 Request failed -> #08 Show error / retry | Next | FAILURE

#05 tickets + returns decision -> #09 Arrange replacement fulfillment | Proposed exchange extension | PLANNED

#09 Arrange replacement fulfillment -> #06 Request status visible | Proposed outcome | PLANNED

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: No automatic replacement order creation, replacement stock reservation or reverse pickup execution was found in the reviewed return request and approval paths. Those steps are Planned.

Refund processing

Architecture overview. Readable node and connection sheets follow.



Refund processing

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Owner requests refund

Payments UI verifies provider payment before refunding.

src/app/admin/(control-center)/payments/page.tsx:1

Next -> #2 (p2)

#02 / HTTP Request / Implemented

POST admin payments

Owner authorization, origin, rate limit and refund input checks.

src/app/api/admin/payments/route.ts:50

Next -> #3 (p2)

#03 / IF / Switch / Implemented

Refundable captured payment?

Verify provider capture, currency, amount, remaining refundable balance and idempotent refund identity.

src/server/razorpay-refunds.ts:100

Yes / accepted -> #4 (p2); No / rejected -> #10 (p4)

#04 / Database Query / Implemented

Record Processing refund

Embedded orders.refunds[] and audit entry record the intent before provider execution.

src/server/razorpay-refunds.ts:408

Next -> #5 (p3)

Refund processing

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / External API / Implemented

Razorpay Refunds API

Gateway may report processed, failed or still processing; ambiguous responses remain unresolved. Runtime configuration and connectivity are Unverified.

src/server/razorpay-refunds.ts:430

Next -> #6 (p3)

#06 / Webhook / Implemented

Signed refund webhook

Verified Razorpay refund event reaches processRefundWebhook.

src/app/api/webhooks/[provider]/route.ts:13

Next -> #7 (p3)

#07 / Backend Service / Implemented

Reconcile remote refund

Refund identity, order association, amount and event status are validated.

src/server/razorpay-refunds.ts:561

Next -> #8 (p3)

#08 / Database Query / Implemented

Refunded / Failed / Processing

Persist only provider-confirmed status; audit and cancellation reconciliation follow.

src/server/razorpay-refunds.ts:284

Next -> #9 (p4)

Refund processing

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Provider refund status

Owner and account views use recorded provider refund status.

src/app/admin/(control-center)/payments/page.tsx:1

END: no outgoing connection

#10 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #11 (p4)

#11 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Refund processing

Complete directed connection register / no omitted success, failure or proposed edges

#01 Owner requests refund -> #02 POST admin payments | Next

#02 POST admin payments -> #03 Refundable captured payment? | Next

#03 Refundable captured payment? -> #04 Record Processing refund | Yes / accepted

#04 Record Processing refund -> #05 Razorpay Refunds API | Next

#05 Razorpay Refunds API -> #06 Signed refund webhook | Next

#06 Signed refund webhook -> #07 Reconcile remote refund | Next

#07 Reconcile remote refund -> #08 Refunded / Failed / Processing | Next

#08 Refunded / Failed / Processing -> #09 Provider refund status | Next

#03 Refundable captured payment? -> #10 Request failed | No / rejected | FAILURE

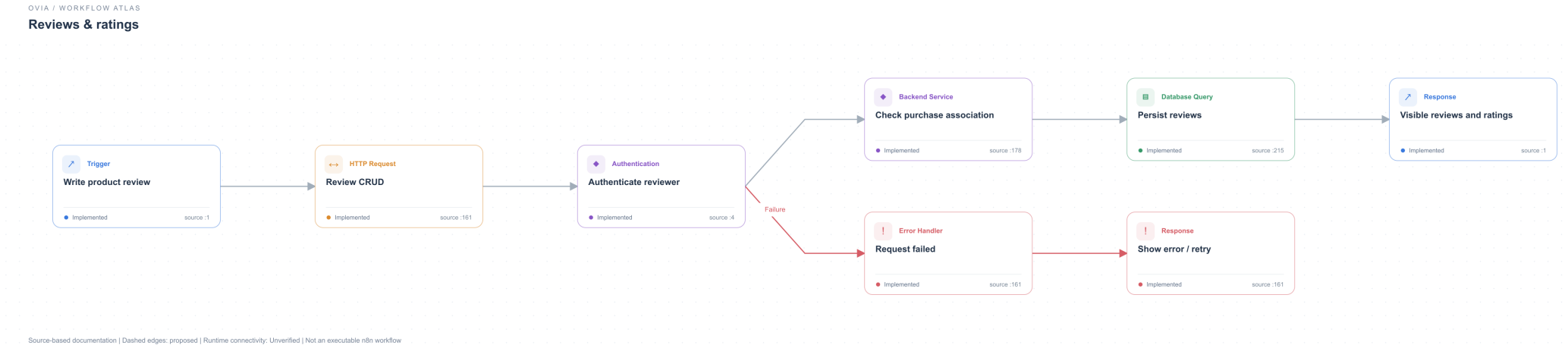
#10 Request failed -> #11 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: Manual returns approval is separate from gateway refund execution. Automated PayPal and Stripe refund execution was not found; a future implementation is Planned.

Reviews & ratings

Architecture overview. Readable node and connection sheets follow.



Reviews & ratings

Readable detail sheet 1/2. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Write product review

Review UI displays published ratings and submits customer content.

src/components/product-reviews.tsx:1

Next -> #2 (p2)

#02 / HTTP Request / Implemented

Review CRUD

Schemas validate rating, comment, aspect scores and attachment metadata.

src/app/api/v1/reviews/route.ts:161

Next -> #3 (p2)

#03 / Authentication / Implemented

Authenticate reviewer

Mutation requires currentUser; edit and delete enforce author ownership.

src/app/api/v1/reviews/route.ts:4

Next -> #4 (p2); Failure -> #7 (p3)

#04 / Backend Service / Implemented

Check purchase association

Stored orders are searched to associate product SKU with a real purchase; verified flag reflects evidence.

src/app/api/v1/reviews/route.ts:178

Next -> #5 (p3)

Reviews & ratings

Readable detail sheet 2/2. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

Persist reviews

reviews holds userId, optional orderId, productSku, productSlug, rating, status and verified.

src/app/api/v1/reviews/route.ts:215

Next -> #6 (p3)

#06 / Response / Implemented

Visible reviews and ratings

Public review read path filters approved reviews.

src/components/product-reviews.tsx:1

END: no outgoing connection

#07 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/app/api/v1/reviews/route.ts:161

Next -> #8 (p3)

#08 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/app/api/v1/reviews/route.ts:161

END: no outgoing connection

Reviews & ratings

Complete directed connection register / no omitted success, failure or proposed edges

#01 Write product review -> #02 Review CRUD | Next

#02 Review CRUD -> #03 Authenticate reviewer | Next

#03 Authenticate reviewer -> #04 Check purchase association | Next

#04 Check purchase association -> #05 Persist reviews | Next

#05 Persist reviews -> #06 Visible reviews and ratings | Next

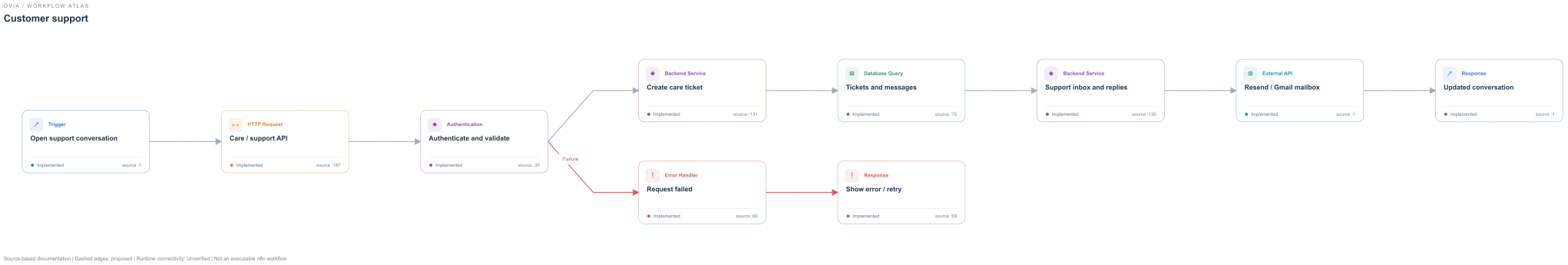
#03 Authenticate reviewer -> #07 Request failed | Failure | FAILURE

#07 Request failed -> #08 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Customer support

Architecture overview. Readable node and connection sheets follow.



Customer support

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Open support conversation

Customer care interface sends tickets, reads conversations and uploads attachments.

src/components/account-support.tsx:1

Next -> #2 (p2)

#02 / HTTP Request / Implemented

Care / support API

Authenticated care path creates structured tickets. /api/v1/support is a separate legacy ticket path.

src/app/api/v1/care/[[...path]]/route.ts:187

Next -> #3 (p2)

#03 / Authentication / Implemented

Authenticate and validate

Care viewer must be signed in; schema and rate limit apply to mutations.

src/app/api/v1/care/[[...path]]/route.ts:30

Next -> #4 (p2); Failure -> #9 (p4)

#04 / Backend Service / Implemented

Create care ticket

Record request and initial support conversation; order-linked requests validate ownership.

src/server/care-order-requests.ts:131

Next -> #5 (p3)

Customer support

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

Tickets and messages

tickets, support_conversations, support_messages and attachment metadata store conversation state.

src/server/support-conversations.ts:70

Next -> #6 (p3)

#06 / Backend Service / Implemented

Support inbox and replies

Owner/support inbox manages replies, status and mail thread linkage.

src/server/support-inbox.ts:130

Next -> #7 (p3)

#07 / External API / Implemented

Resend / Gmail mailbox

Gmail OAuth mailbox sync and Resend outbound delivery are implemented code paths. Runtime configuration and connectivity are Unverified.

src/server/support-gmail.ts:1

Next -> #8 (p3)

#08 / Response / Implemented

Updated conversation

Customer sees support replies; realtime notifications have a separate transport service.

src/components/account-support.tsx:1

END: no outgoing connection

Customer support

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #10 (p4)

#10 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Customer support

Complete directed connection register / no omitted success, failure or proposed edges

#01 Open support conversation -> #02 Care / support API | Next

#02 Care / support API -> #03 Authenticate and validate | Next

#03 Authenticate and validate -> #04 Create care ticket | Next

#04 Create care ticket -> #05 Tickets and messages | Next

#05 Tickets and messages -> #06 Support inbox and replies | Next

#06 Support inbox and replies -> #07 Resend / Gmail mailbox | Next

#07 Resend / Gmail mailbox -> #08 Updated conversation | Next

#03 Authenticate and validate -> #09 Request failed | Failure | FAILURE

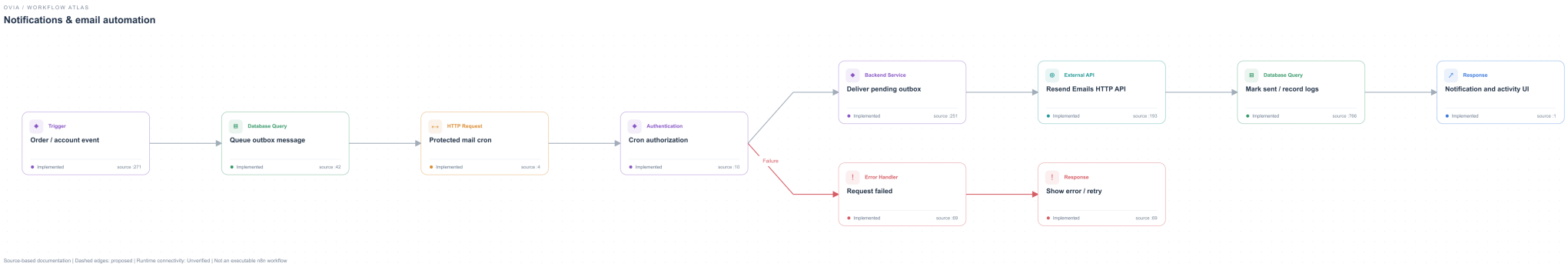
#09 Request failed -> #10 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: Care tickets and legacy support_tickets are distinct collections. Conversation IDs encode the source collection; do not conflate them.

Notifications & email automation

Architecture overview. Readable node and connection sheets follow.



Notifications & email automation

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Order / account event

Account verification, recovery, confirmed orders and shipping events trigger relevant mail routines.

src/server/mail.ts:271

Next -> #2 (p2)

#02 / Database Query / Implemented

Queue outbox message

Outbox IDs and existing-record checks prevent duplicate messages.

src/server/mail.ts:42

Next -> #3 (p2)

#03 / HTTP Request / Implemented

Protected mail cron

Bearer CRON_SECRET is required; no secret value is exported.

src/app/api/cron/mail/route.ts:4

Next -> #4 (p2)

#04 / Authentication / Implemented

Cron authorization

Missing or incorrect bearer token is rejected.

src/app/api/cron/mail/route.ts:10

Next -> #5 (p3); Failure -> #9 (p4)

Notifications & email automation

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Backend Service / Implemented

Deliver pending outbox

Read pending messages, send and mark sent only after successful delivery.

src/server/mail.ts:251

Next -> #6 (p3)

#06 / External API / Implemented

Resend Emails HTTP API

Account and order mail HTTP integration; retries and failure status remain explicit. Runtime configuration and connectivity are Unverified.

src/server/mail.ts:193

Next -> #7 (p3)

#07 / Database Query / Implemented

Mark sent / record logs

Notification studio has notification_logs and deduplication; account events write account_notifications.

src/server/notification-service.ts:766

Next -> #8 (p3)

#08 / Response / Implemented

Notification and activity UI

Account activity reads notifications; admin notification studio reads delivery logs.

src/components/account-activity-feed.tsx:1

END: no outgoing connection

Notifications & email automation

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #10 (p4)

#10 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Notifications & email automation

Complete directed connection register / no omitted success, failure or proposed edges

#01 Order / account event -> #02 Queue outbox message | Next

#02 Queue outbox message -> #03 Protected mail cron | Next

#03 Protected mail cron -> #04 Cron authorization | Next

#04 Cron authorization -> #05 Deliver pending outbox | Next

#05 Deliver pending outbox -> #06 Resend Emails HTTP API | Next

#06 Resend Emails HTTP API -> #07 Mark sent / record logs | Next

#07 Mark sent / record logs -> #08 Notification and activity UI | Next

#04 Cron authorization -> #09 Request failed | Failure | FAILURE

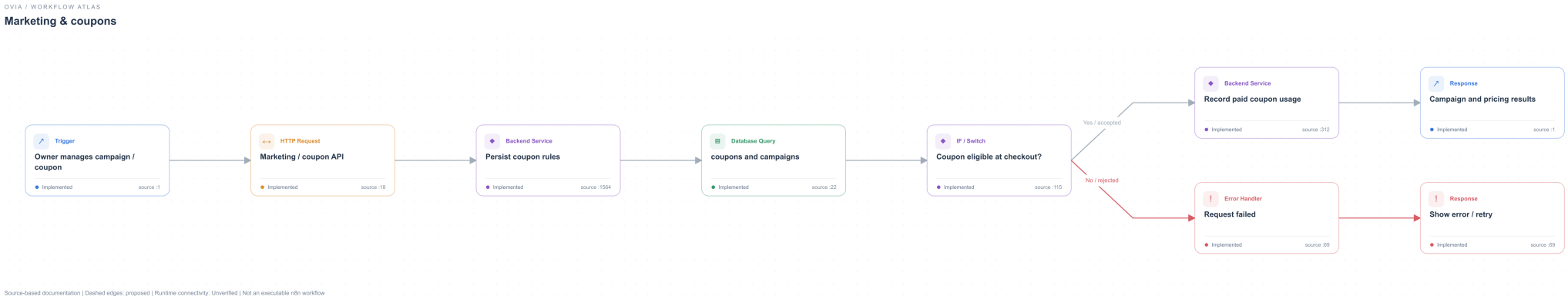
#09 Request failed -> #10 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: Outbox delivery and notification studio dispatch are related but separate service paths; not every outbox send creates a notification_logs entry.

Marketing & coupons

Architecture overview. Readable node and connection sheets follow.



Marketing & coupons

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Owner manages campaign / coupon

Marketing and coupon admin interfaces.

src/app/admin/(control-center)/marketing/page.tsx:1

Next -> #2 (p2)

#02 / HTTP Request / Implemented

Marketing / coupon API

Owner-authorized CRUD; campaign routes collect storefront activity.

src/app/api/admin/coupons/route.ts:18

Next -> #3 (p2)

#03 / Backend Service / Implemented

Persist coupon rules

Validate discount, eligibility, usage and expiration settings.

src/server/admin-service.ts:1564

Next -> #4 (p2)

#04 / Database Query / Implemented

coupons and campaigns

Campaign events and order campaignId support attribution; coupons store pricing rules.

src/server/campaigns.ts:22

Next -> #5 (p3)

Marketing & coupons

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / IF / Switch / Implemented

Coupon eligible at checkout?

Server reads current coupon and prior customer orders to apply allowed pricing.

src/server/checkout-pricing.ts:115

Yes / accepted -> #6 (p3); No / rejected -> #8 (p3)

#06 / Backend Service / Implemented

Record paid coupon usage

Confirmed order usage is recorded; pending checkout must not consume paid usage.

src/server/checkout-pricing.ts:312

Next -> #7 (p3)

#07 / Response / Implemented

Campaign and pricing results

Marketing UI and checkout display persisted campaign and discount outcomes.

src/app/admin/(control-center)/marketing/page.tsx:1

END: no outgoing connection

#08 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #9 (p4)

Marketing & coupons

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Marketing & coupons

Complete directed connection register / no omitted success, failure or proposed edges

#01 Owner manages campaign / coupon -> #02 Marketing / coupon API | Next

#02 Marketing / coupon API -> #03 Persist coupon rules | Next

#03 Persist coupon rules -> #04 coupons and campaigns | Next

#04 coupons and campaigns -> #05 Coupon eligible at checkout? | Next

#05 Coupon eligible at checkout? -> #06 Record paid coupon usage | Yes / accepted

#06 Record paid coupon usage -> #07 Campaign and pricing results | Next

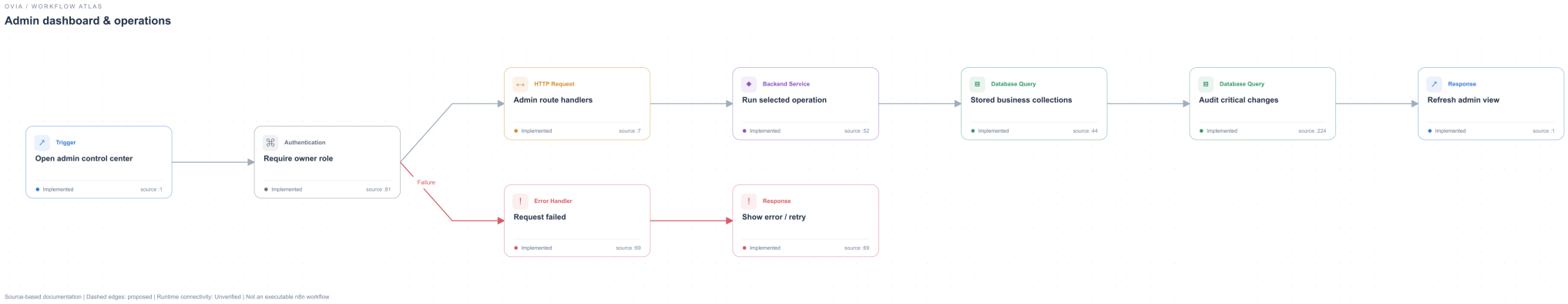
#05 Coupon eligible at checkout? -> #08 Request failed | No / rejected | FAILURE

#08 Request failed -> #09 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Admin dashboard & operations

Architecture overview. Readable node and connection sheets follow.



Admin dashboard & operations

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Open admin control center

Admin shell groups operations and navigation.

src/components/admin/admin-shell.tsx:1

Next -> #2 (p2)

#02 / Authentication / Implemented

Require owner role

Server components redirect missing sessions, unauthorized roles or database unavailability.

src/server/admin-auth.ts:81

Next -> #3 (p2); Failure -> #8 (p3)

#03 / HTTP Request / Implemented

Admin route handlers

Each API independently authorizes the owner session; no central middleware is invented.

src/app/api/admin/dashboard/route.ts:7

Next -> #4 (p2)

#04 / Backend Service / Implemented

Run selected operation

Actual services cover products, inventory, orders, customers, payments, returns, support and settings.

src/server/admin-service.ts:52

Next -> #5 (p3)

Admin dashboard & operations

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

Stored business collections

Repository queries and transactional mutations use native MongoDB driver.

src/server/db.ts:44

Next -> #6 (p3)

#06 / Database Query / Implemented

Audit critical changes

Critical owner operations record actor, action, resource, resourceId and details.

src/server/admin-auth.ts:224

Next -> #7 (p3)

#07 / Response / Implemented

Refresh admin view

Admin client refresh hooks request current data.

src/components/admin/use-live-admin-refresh.ts:1

END: no outgoing connection

#08 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #9 (p4)

Admin dashboard & operations

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Admin dashboard & operations

Complete directed connection register / no omitted success, failure or proposed edges

#01 Open admin control center -> #02 Require owner role | Next

#02 Require owner role -> #03 Admin route handlers | Next

#03 Admin route handlers -> #04 Run selected operation | Next

#04 Run selected operation -> #05 Stored business collections | Next

#05 Stored business collections -> #06 Audit critical changes | Next

#06 Audit critical changes -> #07 Refresh admin view | Next

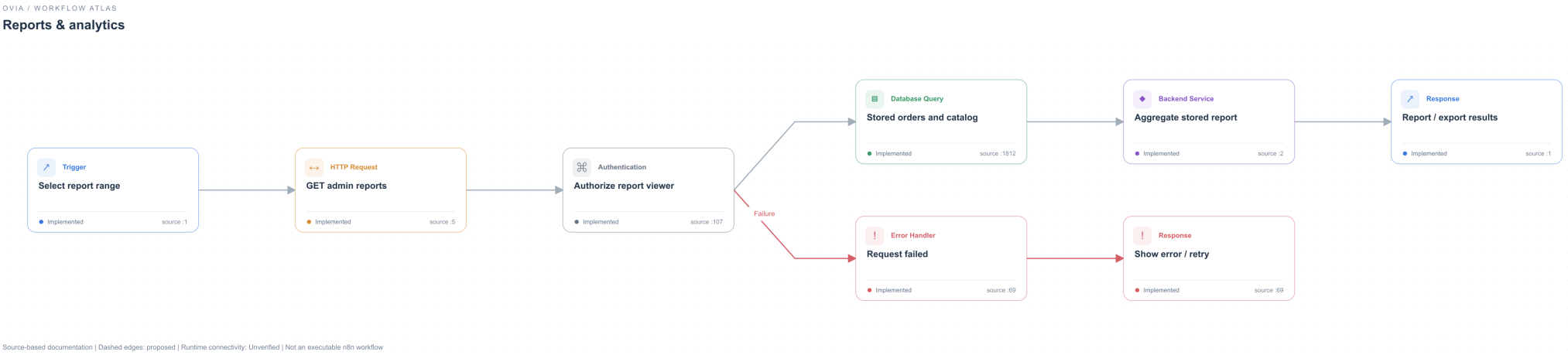
#02 Require owner role -> #08 Request failed | Failure | FAILURE

#08 Request failed -> #09 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Reports & analytics

Architecture overview. Readable node and connection sheets follow.



Reports & analytics

Readable detail sheet 1/2. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Select report range

Owner reports page selects report type and range.

src/app/admin/(control-center)/reports/page.tsx:1

Next -> #2 (p2)

#02 / HTTP Request / Implemented

GET admin reports

authorizeAdminApi precedes getAdminReports.

src/app/api/admin/reports/route.ts:5

Next -> #3 (p2)

#03 / Authentication / Implemented

Authorize report viewer

Only authorized owner sessions can read admin reports.

src/server/admin-auth.ts:107

Next -> #4 (p2); Failure -> #7 (p3)

#04 / Database Query / Implemented

Stored orders and catalog

Orders plus full catalog feed buildStoredSalesReport; no synthetic revenue is claimed.

src/server/admin-service.ts:1812

Next -> #5 (p3)

Reports & analytics

Readable detail sheet 2/2. Node numbers and page references preserve every directed connection.

#05 / Backend Service / Implemented

Aggregate stored report

Report calculations are imported from admin date-range helpers.

src/server/admin-service.ts:2

Next -> #6 (p3)

#06 / Response / Implemented

Report / export results

Reports UI presents calculated data; admin dashboard and campaign analytics are separate routes.

src/app/admin/(control-center)/reports/page.tsx:1

END: no outgoing connection

#07 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #8 (p3)

#08 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Reports & analytics

Complete directed connection register / no omitted success, failure or proposed edges

#01 Select report range -> #02 GET admin reports | Next

#02 GET admin reports -> #03 Authorize report viewer | Next

#03 Authorize report viewer -> #04 Stored orders and catalog | Next

#04 Stored orders and catalog -> #05 Aggregate stored report | Next

#05 Aggregate stored report -> #06 Report / export results | Next

#03 Authorize report viewer -> #07 Request failed | Failure | FAILURE

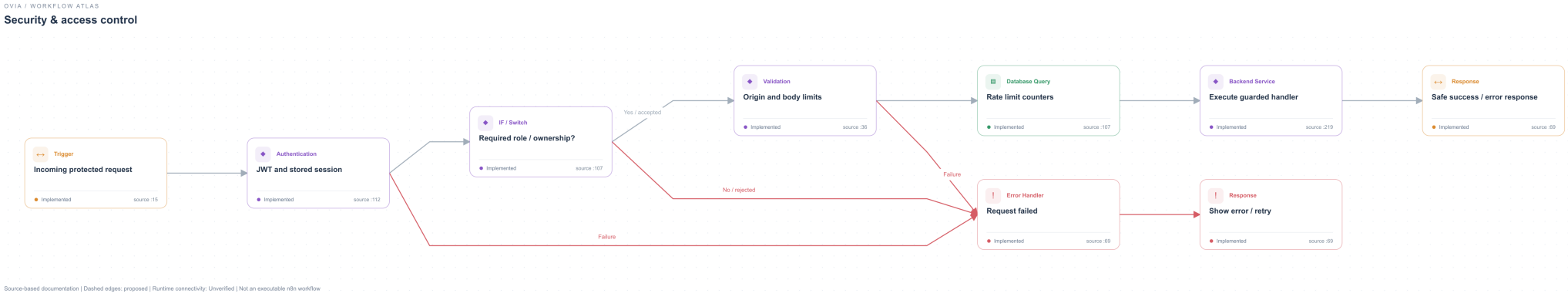
#07 Request failed -> #08 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: AI copilot is an implemented optional DeepSeek request path. Model quality and production connectivity are Unverified; it does not replace stored sales calculations.

Security & access control

Architecture overview. Readable node and connection sheets follow.



Security & access control

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Incoming protected request

Applicable mutation handlers apply guards locally.

src/server/http.ts:15

Next -> #2 (p2)

#02 / Authentication / Implemented

JWT and stored session

Check HS256, issuer, audience, jti hash, session expiry and verified user record.

src/server/auth.ts:112

Next -> #3 (p2); Failure -> #8 (p3)

#03 / IF / Switch / Implemented

Required role / ownership?

Owner admin APIs use normalized OWNER/ADMIN; legacy staff routes have different literal role checks.

src/server/admin-auth.ts:107

Yes / accepted -> #4 (p2); No / rejected -> #8 (p3)

#04 / Validation / Implemented

Origin and body limits

Origin allowlist and streamed body cap are helpers explicitly called by participating routes.

src/server/http.ts:36

Next -> #5 (p3); Failure -> #8 (p3)

Security & access control

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

Rate limit counters

limits collection tracks scoped counts in a sixty-second window; exceeding threshold returns 429.

src/server/http.ts:107

Next -> #6 (p3)

#06 / Backend Service / Implemented

Execute guarded handler

Handlers parse Zod input and authorize the applicable resource before service mutations.

src/app/api/v1/[[...path]]/route.ts:219

Next -> #7 (p3)

#07 / Response / Implemented

Safe success / error response

api() maps Zod to 400, duplicate records to 409, ApiError to its code, unknown service errors to 503; redacts environment secrets.

src/server/http.ts:69

END: no outgoing connection

#08 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #9 (p4)

Security & access control

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Security & access control

Complete directed connection register / no omitted success, failure or proposed edges

#01 Incoming protected request -> #02 JWT and stored session | Next

#02 JWT and stored session -> #03 Required role / ownership? | Next

#03 Required role / ownership? -> #04 Origin and body limits | Yes / accepted

#04 Origin and body limits -> #05 Rate limit counters | Next

#05 Rate limit counters -> #06 Execute guarded handler | Next

#06 Execute guarded handler -> #07 Safe success / error response | Next

#02 JWT and stored session -> #08 Request failed | Failure | FAILURE

#03 Required role / ownership? -> #08 Request failed | No / rejected | FAILURE

#04 Origin and body limits -> #08 Request failed | Failure | FAILURE

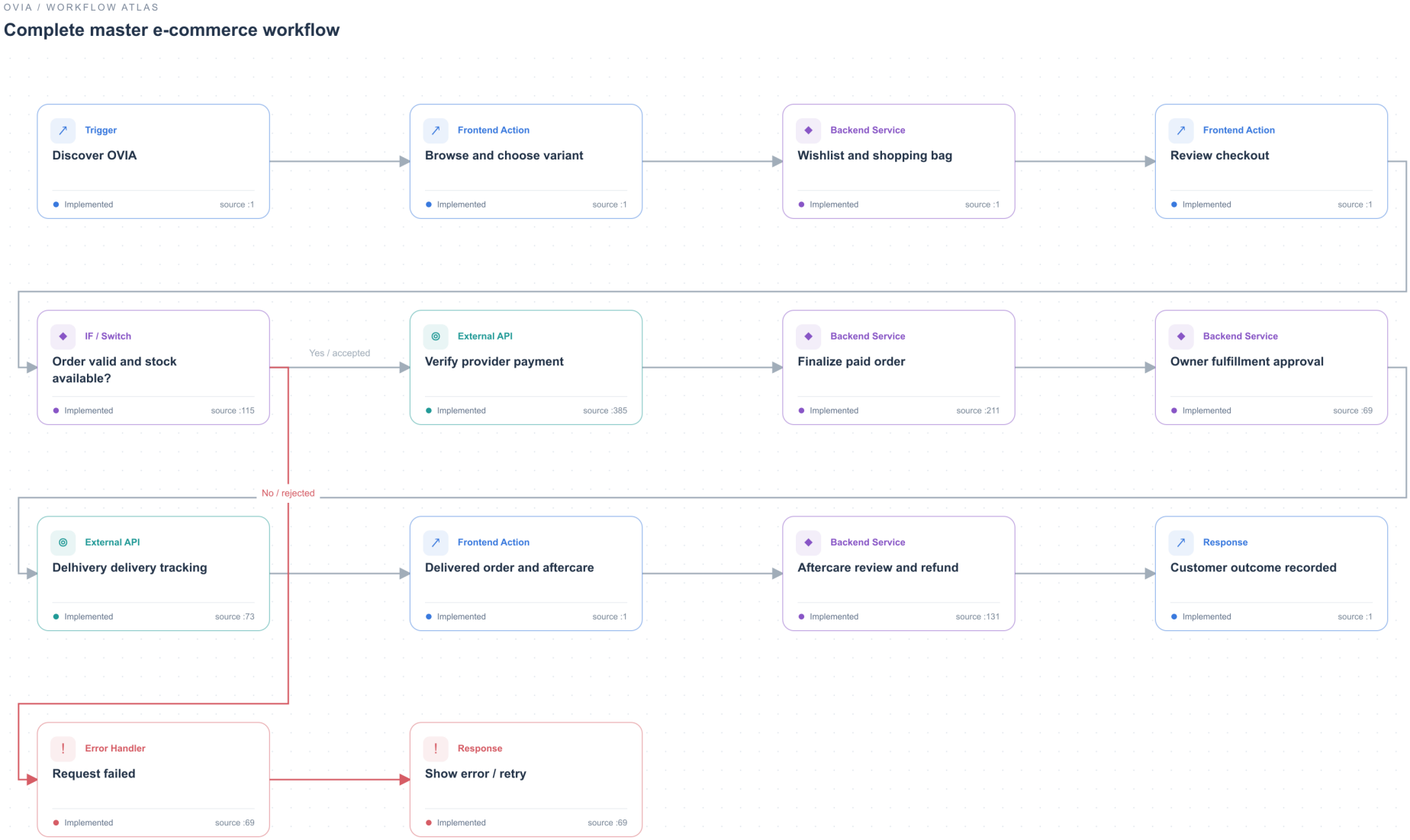
#08 Request failed -> #09 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: These protections are not universal middleware. API inventory lists checks found per handler; several routes use custom response/error wrappers. Broad origin allowances and inconsistent legacy role checks are documented review items, not silently corrected.

Complete master e-commerce workflow

Architecture overview. Readable node and connection sheets follow.



Complete master e-commerce workflow

Readable detail sheet 1/4. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Discover OVIA

Home, navigation and bag / luggage storefront.

src/app/page.tsx:1

Next -> #2 (p2)

#02 / Frontend Action / Implemented

Browse and choose variant

Product listing, detail, search and filter flows.

src/components/product-detail.tsx:1

Next -> #3 (p2)

#03 / Backend Service / Implemented

Wishlist and shopping bag

Frontend provider sends cart and wishlist mutations.

src/components/store-provider.tsx:1

Next -> #4 (p2)

#04 / Frontend Action / Implemented

Review checkout

Address, PIN serviceability, shipping method and quote.

src/app/checkout/page.tsx:1

Next -> #5 (p3)

Complete master e-commerce workflow

Readable detail sheet 2/4. Node numbers and page references preserve every directed connection.

#05 / IF / Switch / Implemented

Order valid and stock available?

Idempotent order and stock reservation transaction.

src/server/commerce.ts:115

Yes / accepted -> #6 (p3); No / rejected -> #13 (p5)

#06 / External API / Implemented

Verify provider payment

Razorpay secure checkout; PayPal sandbox is an alternate flow. Runtime configuration and connectivity are Unverified.

src/server/razorpay-checkout.ts:385

Next -> #7 (p3)

#07 / Backend Service / Implemented

Finalize paid order

Confirmed paid state, replay event and mail queue.

src/server/commerce.ts:211

Next -> #8 (p3)

#08 / Backend Service / Implemented

Owner fulfillment approval

Verified payment precedes courier shipment and pickup.

src/server/logistics.ts:69

Next -> #9 (p4)

Complete master e-commerce workflow

Readable detail sheet 3/4. Node numbers and page references preserve every directed connection.

#09 / External API / Implemented

Delhivery delivery tracking

Courier API / authenticated webhook updates stored shipping. Runtime configuration and connectivity are Unverified.

src/lib/shipping/service.ts:73

Next -> #10 (p4)

#10 / Frontend Action / Implemented

Delivered order and aftercare

Customer tracks delivery, submits reviews or opens cancellation / return / refund requests.

src/components/account-order-details.tsx:1

Next -> #11 (p4)

#11 / Backend Service / Implemented

Aftercare review and refund

Tickets are reviewed; gateway refunds follow the separate authorized refund flow.

src/server/care-order-requests.ts:131

Next -> #12 (p4)

#12 / Response / Implemented

Customer outcome recorded

Display delivery, review request, cancellation and provider refund evidence.

src/components/account-orders-returns.tsx:1

END: no outgoing connection

Complete master e-commerce workflow

Readable detail sheet 4/4. Node numbers and page references preserve every directed connection.

#13 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #14 (p5)

#14 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Complete master e-commerce workflow

Complete directed connection register / no omitted success, failure or proposed edges

#01 Discover OVIA -> #02 Browse and choose variant | Next

#02 Browse and choose variant -> #03 Wishlist and shopping bag | Next

#03 Wishlist and shopping bag -> #04 Review checkout | Next

#04 Review checkout -> #05 Order valid and stock available? | Next

#05 Order valid and stock available? -> #06 Verify provider payment | Yes / accepted

#06 Verify provider payment -> #07 Finalize paid order | Next

#07 Finalize paid order -> #08 Owner fulfillment approval | Next

#08 Owner fulfillment approval -> #09 Delhivery delivery tracking | Next

#09 Delhivery delivery tracking -> #10 Delivered order and aftercare | Next

#10 Delivered order and aftercare -> #11 Aftercare review and refund | Next

#11 Aftercare review and refund -> #12 Customer outcome recorded | Next

#05 Order valid and stock available? -> #13 Request failed | No / rejected | FAILURE

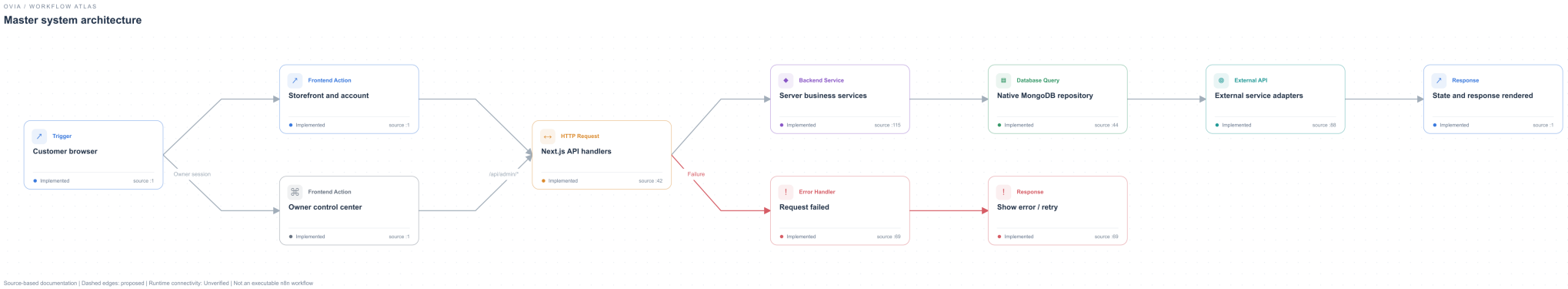
#13 Request failed -> #14 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: This master is a process composition of the individual evidence-mapped workflows, not one executable handler.

Master system architecture

Architecture overview. Readable node and connection sheets follow.



Master system architecture

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Customer browser

Next.js storefront shell with customer-facing pages and client components.

src/app/layout.tsx:1

Next -> #2 (p2); Owner session -> #10 (p4)

#02 / Frontend Action / Implemented

Storefront and account

Catalog, bag, checkout and account state coordinate client requests.

src/components/store-provider.tsx:1

Next -> #3 (p2)

#03 / HTTP Request / Implemented

Next.js API handlers

86 route files, including explicit dispatch branches and provider webhooks.

src/app/api/v1/[[...path]]/route.ts:42

Next -> #4 (p2); Failure -> #8 (p3)

#04 / Backend Service / Implemented

Server business services

Commerce, accounts, logistics, mail, support, marketing and reports.

src/server/commerce.ts:115

Next -> #5 (p3)

Master system architecture

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

Native MongoDB repository

MongoClient pool; repository access and withTransaction for coordinated writes.

src/server/db.ts:44

Next -> #6 (p3)

#06 / External API / Implemented

External service adapters

Razorpay, PayPal sandbox, Delhivery, Resend, Algolia, OAuth, Gmail, translation and DeepSeek. Runtime configuration and connectivity are Unverified.

src/server/integrations-service.ts:88

Next -> #7 (p3)

#07 / Response / Implemented

State and response rendered

Safe route responses update customer and owner views.

src/components/store-provider.tsx:1

END: no outgoing connection

#08 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #9 (p4)

Master system architecture

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

#10 / Frontend Action / Implemented

Owner control center

Server-protected owner operations and API-specific authorization.

src/components/admin/admin-shell.tsx:1

/api/admin/* -> #3 (p2)

Master system architecture

Complete directed connection register / no omitted success, failure or proposed edges

#01 Customer browser -> #02 Storefront and account | Next

#02 Storefront and account -> #03 Next.js API handlers | Next

#03 Next.js API handlers -> #04 Server business services | Next

#04 Server business services -> #05 Native MongoDB repository | Next

#05 Native MongoDB repository -> #06 External service adapters | Next

#06 External service adapters -> #07 State and response rendered | Next

#03 Next.js API handlers -> #08 Request failed | Failure | FAILURE

#08 Request failed -> #09 Show error / retry | Next | FAILURE

#01 Customer browser -> #10 Owner control center | Owner session

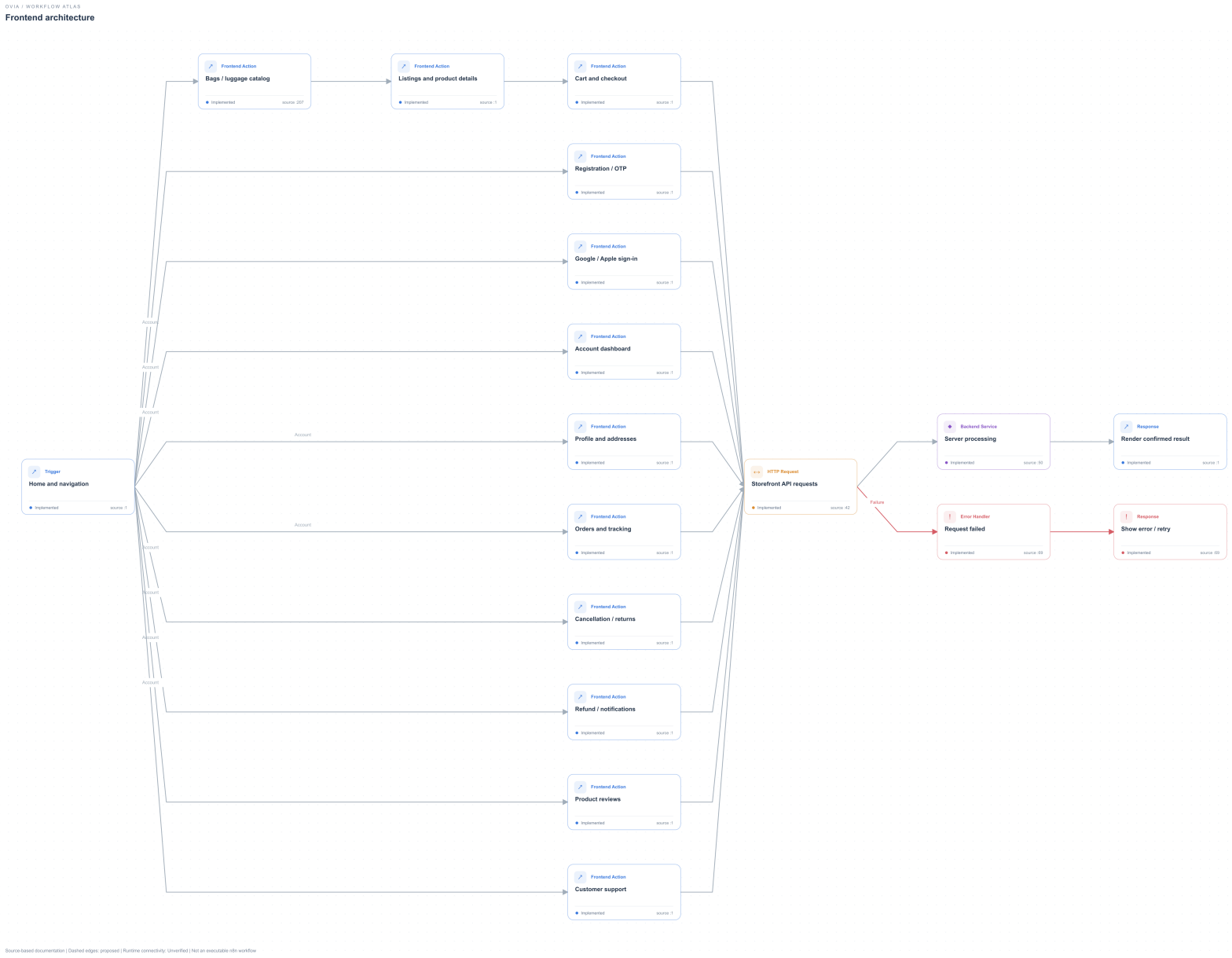
#10 Owner control center -> #03 Next.js API handlers | /api/admin/*

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: MongoDB Atlas deployment topology is Unverified. The code proves a MongoDB connection and database selection, not specific cluster tiers or regions.

Frontend architecture

Architecture overview. Readable node and connection sheets follow.



Frontend architecture

Readable detail sheet 1/5. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Home and navigation

Homepage and shared header / navigation.

src/app/page.tsx:1

Next -> #2 (p2); Account -> #10 (p4); Account -> #11 (p4); Account -> #12 (p4); Account -> #13 (p5); Account -> #14 (p5); Account -> #15 (p5); Account -> #16 (p5); Account -> #17 (p6); Account -> #18 (p6)

#02 / Frontend Action / Implemented

Bags / luggage catalog

Catchall storefront resolves bags, luggage, categories and collection routes.

src/app/[...segments]/page.tsx:207

Next -> #3 (p2)

#03 / Frontend Action / Implemented

Listings and product details

Catalog facets and /products/[slug] detail.

src/components/catalog-view.tsx:1

Next -> #4 (p2)

#04 / Frontend Action / Implemented

Cart and checkout

Store provider and cart page send server cart / checkout requests.

src/components/store-provider.tsx:1

Next -> #5 (p3)

Frontend architecture

Readable detail sheet 2/5. Node numbers and page references preserve every directed connection.

#05 / HTTP Request / Implemented

Storefront API requests

Actual client/server routes documented in the route inventory.

src/app/api/v1/[[...path]]/route.ts:42

Next -> #6 (p3); Failure -> #8 (p3)

#06 / Backend Service / Implemented

Server processing

Services return current catalog, bag, quote and order data.

src/server/commerce.ts:50

Next -> #7 (p3)

#07 / Response / Implemented

Render confirmed result

State updates follow API results, with loading and error UI.

src/components/store-provider.tsx:1

END: no outgoing connection

#08 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #9 (p4)

Frontend architecture

Readable detail sheet 3/5. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

#10 / Frontend Action / Implemented

Registration / OTP

Customer account section backed by actual account, care, order or review APIs. Address book is a separate component.

src/components/auth-experience.tsx:1

Next -> #5 (p3)

#11 / Frontend Action / Implemented

Google / Apple sign-in

Customer account section backed by actual account, care, order or review APIs. Address book is a separate component.

src/components/social-sign-in-buttons.tsx:1

Next -> #5 (p3)

#12 / Frontend Action / Implemented

Account dashboard

Customer account section backed by actual account, care, order or review APIs. Address book is a separate component.

src/components/account-dashboard.tsx:1

Next -> #5 (p3)

Frontend architecture

Readable detail sheet 4/5. Node numbers and page references preserve every directed connection.

#13 / Frontend Action / Implemented

Profile and addresses

Customer account section backed by actual account, care, order or review APIs. Address book is a separate component.

src/components/customer-profile.tsx:1

Next -> #5 (p3)

#14 / Frontend Action / Implemented

Orders and tracking

Customer account section backed by actual account, care, order or review APIs. Address book is a separate component.

src/components/account-orders-portal.tsx:1

Next -> #5 (p3)

#15 / Frontend Action / Implemented

Cancellation / returns

Customer account section backed by actual account, care, order or review APIs. Address book is a separate component.

src/components/account-orders-returns.tsx:1

Next -> #5 (p3)

#16 / Frontend Action / Implemented

Refund / notifications

Customer account section backed by actual account, care, order or review APIs. Address book is a separate component.

src/components/account-activity-feed.tsx:1

Next -> #5 (p3)

Frontend architecture

Readable detail sheet 5/5. Node numbers and page references preserve every directed connection.

#17 / Frontend Action / Implemented

Product reviews

Customer account section backed by actual account, care, order or review APIs. Address book is a separate component.

src/components/account-reviews.tsx:1

Next -> #5 (p3)

#18 / Frontend Action / Implemented

Customer support

Customer account section backed by actual account, care, order or review APIs. Address book is a separate component.

src/components/account-support.tsx:1

Next -> #5 (p3)

Frontend architecture

Complete directed connection register / no omitted success, failure or proposed edges

#01 Home and navigation -> #02 Bags / luggage catalog | Next

#02 Bags / luggage catalog -> #03 Listings and product details | Next

#03 Listings and product details -> #04 Cart and checkout | Next

#04 Cart and checkout -> #05 Storefront API requests | Next

#05 Storefront API requests -> #06 Server processing | Next

#06 Server processing -> #07 Render confirmed result | Next

#05 Storefront API requests -> #08 Request failed | Failure | FAILURE

#08 Request failed -> #09 Show error / retry | Next | FAILURE

#01 Home and navigation -> #10 Registration / OTP | Account

#10 Registration / OTP -> #05 Storefront API requests | Next

#01 Home and navigation -> #11 Google / Apple sign-in | Account

#11 Google / Apple sign-in -> #05 Storefront API requests | Next

#01 Home and navigation -> #12 Account dashboard | Account

#12 Account dashboard -> #05 Storefront API requests | Next

#01 Home and navigation -> #13 Profile and addresses | Account

#13 Profile and addresses -> #05 Storefront API requests | Next

#01 Home and navigation -> #14 Orders and tracking | Account

#14 Orders and tracking -> #05 Storefront API requests | Next

#01 Home and navigation -> #15 Cancellation / returns | Account

#15 Cancellation / returns -> #05 Storefront API requests | Next

#01 Home and navigation -> #16 Refund / notifications | Account

#16 Refund / notifications -> #05 Storefront API requests | Next

Frontend architecture

Directed connection register continued

#01 Home and navigation -> #17 Product reviews | Account

#17 Product reviews -> #05 Storefront API requests | Next

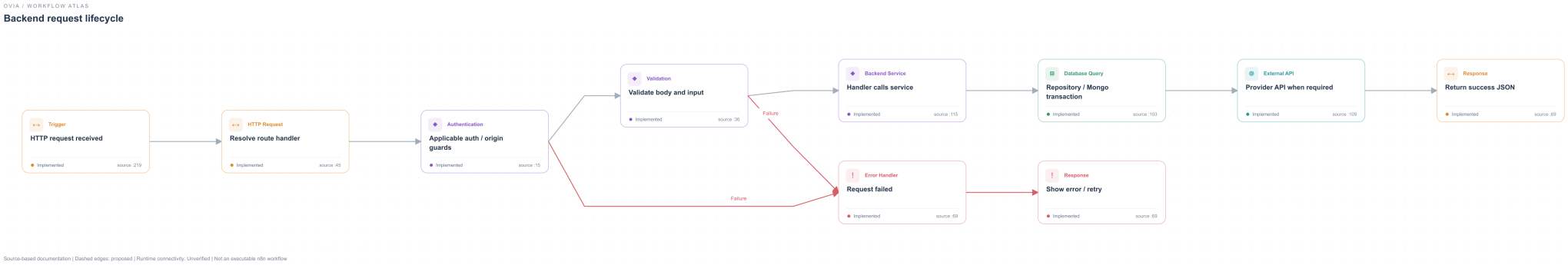
#01 Home and navigation -> #18 Customer support | Account

#18 Customer support -> #05 Storefront API requests | Next

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Backend request lifecycle

Architecture overview. Readable node and connection sheets follow.



Backend request lifecycle

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

HTTP request received

Next.js route receives request and promised path params.

src/app/api/v1/[[...path]]/route.ts:219

Next -> #2 (p2)

#02 / HTTP Request / Implemented

Resolve route handler

App Router file routes plus explicit catchall dispatch.

src/app/api/v1/[[...path]]/route.ts:45

Next -> #3 (p2)

#03 / Authentication / Implemented

Applicable auth / origin guards

Each handler decides which helpers apply; this is a representative guarded mutation lifecycle.

src/server/http.ts:15

Next -> #4 (p2); Failure -> #9 (p4)

#04 / Validation / Implemented

Validate body and input

Streaming body cap and Zod validation where explicitly called.

src/server/http.ts:36

Next -> #5 (p3); Failure -> #9 (p4)

Backend request lifecycle

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Backend Service / Implemented

Handler calls service

Actual business rules and idempotency are in server modules.

src/server/commerce.ts:115

Next -> #6 (p3)

#06 / Database Query / Implemented

Repository / Mongo transaction

Native database operations share a session when required.

src/server/db.ts:103

Next -> #7 (p3)

#07 / External API / Implemented

Provider API when required

External writes are reconciled with durable local state; not all requests call a provider. Runtime configuration and connectivity are Unverified.

src/server/razorpay-checkout.ts:109

Next -> #8 (p3)

#08 / Response / Implemented

Return success JSON

Common wrapper returns {data}; other routes use documented custom response shapes.

src/server/http.ts:69

END: no outgoing connection

Backend request lifecycle

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #10 (p4)

#10 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Backend request lifecycle

Complete directed connection register / no omitted success, failure or proposed edges

#01 HTTP request received -> #02 Resolve route handler | Next

#02 Resolve route handler -> #03 Applicable auth / origin guards | Next

#03 Applicable auth / origin guards -> #04 Validate body and input | Next

#04 Validate body and input -> #05 Handler calls service | Next

#05 Handler calls service -> #06 Repository / Mongo transaction | Next

#06 Repository / Mongo transaction -> #07 Provider API when required | Next

#07 Provider API when required -> #08 Return success JSON | Next

#03 Applicable auth / origin guards -> #09 Request failed | Failure | FAILURE

#04 Validate body and input -> #09 Request failed | Failure | FAILURE

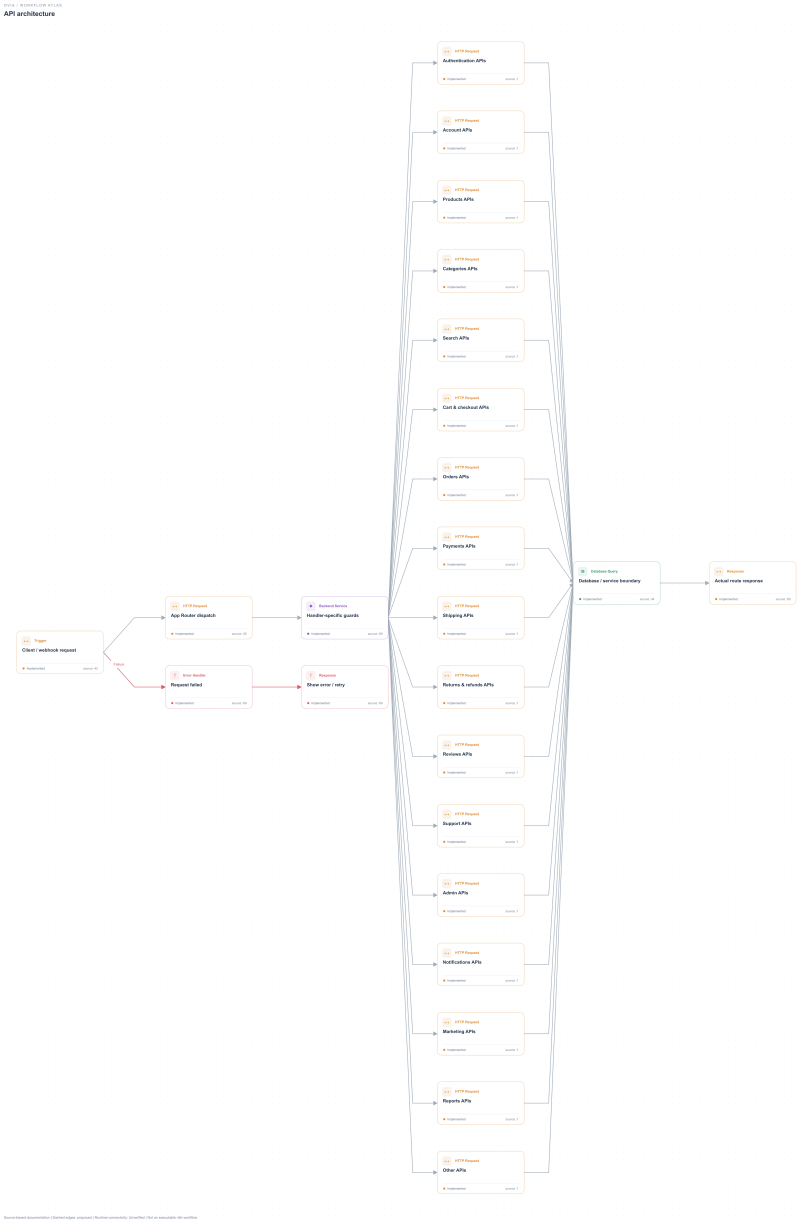
#09 Request failed -> #10 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: Source contains no central src/middleware.ts or src/proxy.ts. Guards are route-local and server-component-local. Logging is selective, not a universal centralized logging pipeline.

API architecture

Architecture overview. Readable node and connection sheets follow.



API architecture

Readable detail sheet 1/6. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Client / webhook request

Customer, owner, cron and provider entry points.

src/app/api/v1/[[...path]]/route.ts:42

Next -> #2 (p2); Failure -> #6 (p3)

#02 / HTTP Request / Implemented

App Router dispatch

Inventory records every exported method from all 86 route files; catchalls have an additional explicit branch inventory.

src/app/api/v1/[[...path]]/route.ts:45

Next -> #3 (p2)

#03 / Backend Service / Implemented

Handler-specific guards

Auth, origin, rate limits, validation and response conventions vary by endpoint.

src/server/http.ts:69

Next -> #8 (p3); Next -> #9 (p4); Next -> #10 (p4); Next -> #11 (p4); Next -> #12 (p4); Next -> #13 (p5); Next -> #14 (p5); Next -> #15 (p5); Next -> #16 (p5); Next -> #17 (p6); Next -> #18 (p6); Next -> #19 (p6); Next -> #20 (p6); Next -> #21 (p7);...

#04 / Database Query / Implemented

Database / service boundary

Imported service symbols and source file evidence are retained per operation.

src/server/db.ts:44

Next -> #5 (p3)

API architecture

Readable detail sheet 2/6. Node numbers and page references preserve every directed connection.

#05 / Response / Implemented

Actual route response

Use the inventory and per-operation graph to inspect the full source-derived lifecycle.

src/server/http.ts:69

END: no outgoing connection

#06 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #7 (p3)

#07 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

#08 / HTTP Request / Implemented

Authentication APIs

5 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/admin/auth/login/route.ts:1

Next -> #4 (p2)

API architecture

Readable detail sheet 3/6. Node numbers and page references preserve every directed connection.

#09 / HTTP Request / Implemented

Account APIs

14 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/v1/account/activity/route.ts:1

Next -> #4 (p2)

#10 / HTTP Request / Implemented

Products APIs

2 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/products/route.ts:1

Next -> #4 (p2)

#11 / HTTP Request / Implemented

Categories APIs

1 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/categories/route.ts:1

Next -> #4 (p2)

#12 / HTTP Request / Implemented

Search APIs

1 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/search/route.ts:1

Next -> #4 (p2)

API architecture

Readable detail sheet 4/6. Node numbers and page references preserve every directed connection.

#13 / HTTP Request / Implemented

Cart & checkout APIs

2 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/v1/checkout/[[...action]]/route.ts:1

Next -> #4 (p2)

#14 / HTTP Request / Implemented

Orders APIs

4 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/v1/bulk-orders/route.ts:1

Next -> #4 (p2)

#15 / HTTP Request / Implemented

Payments APIs

2 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/v1/payments/razorpay/[[...action]]/route.ts:1

Next -> #4 (p2)

#16 / HTTP Request / Implemented

Shipping APIs

3 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/orders/[orderId]/tracking/route.ts:1

Next -> #4 (p2)

API architecture

Readable detail sheet 5/6. Node numbers and page references preserve every directed connection.

#17 / HTTP Request / Implemented

Returns & refunds APIs

1 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/v1/orders/[id]/return/route.ts:1

Next -> #4 (p2)

#18 / HTTP Request / Implemented

Reviews APIs

4 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/v1/reviews/route.ts:1

Next -> #4 (p2)

#19 / HTTP Request / Implemented

Support APIs

11 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/internal/support/sync/route.ts:1

Next -> #4 (p2)

#20 / HTTP Request / Implemented

Admin APIs

70 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/admin/abandoned-carts/route.ts:1

Next -> #4 (p2)

API architecture

Readable detail sheet 6/6. Node numbers and page references preserve every directed connection.

#21 / HTTP Request / Implemented

Notifications APIs

3 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/admin/notifications/route.ts:1

Next -> #4 (p2)

#22 / HTTP Request / Implemented

Marketing APIs

13 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/admin/coupons/route.ts:1

Next -> #4 (p2)

#23 / HTTP Request / Implemented

Reports APIs

2 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/admin/dashboard/route.ts:1

Next -> #4 (p2)

#24 / HTTP Request / Implemented

Other APIs

14 exported file-route operations. See inventory for individual handlers, service symbols and detected guards.

src/app/api/cron/expire/route.ts:1

Next -> #4 (p2)

API architecture

Complete directed connection register / no omitted success, failure or proposed edges

#01 Client / webhook request -> #02 App Router dispatch | Next

#02 App Router dispatch -> #03 Handler-specific guards | Next

#04 Database / service boundary -> #05 Actual route response | Next

#01 Client / webhook request -> #06 Request failed | Failure | FAILURE

#06 Request failed -> #07 Show error / retry | Next | FAILURE

#03 Handler-specific guards -> #08 Authentication APIs | Next

#08 Authentication APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #09 Account APIs | Next

#09 Account APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #10 Products APIs | Next

#10 Products APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #11 Categories APIs | Next

#11 Categories APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #12 Search APIs | Next

#12 Search APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #13 Cart & checkout APIs | Next

#13 Cart & checkout APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #14 Orders APIs | Next

#14 Orders APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #15 Payments APIs | Next

#15 Payments APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #16 Shipping APIs | Next

API architecture

Directed connection register continued

#16 Shipping APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #17 Returns & refunds APIs | Next

#17 Returns & refunds APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #18 Reviews APIs | Next

#18 Reviews APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #19 Support APIs | Next

#19 Support APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #20 Admin APIs | Next

#20 Admin APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #21 Notifications APIs | Next

#21 Notifications APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #22 Marketing APIs | Next

#22 Marketing APIs -> #04 Database / service boundary | Next

#03 Handler-specific guards -> #23 Reports APIs | Next

#23 Reports APIs -> #04 Database / service boundary | Next

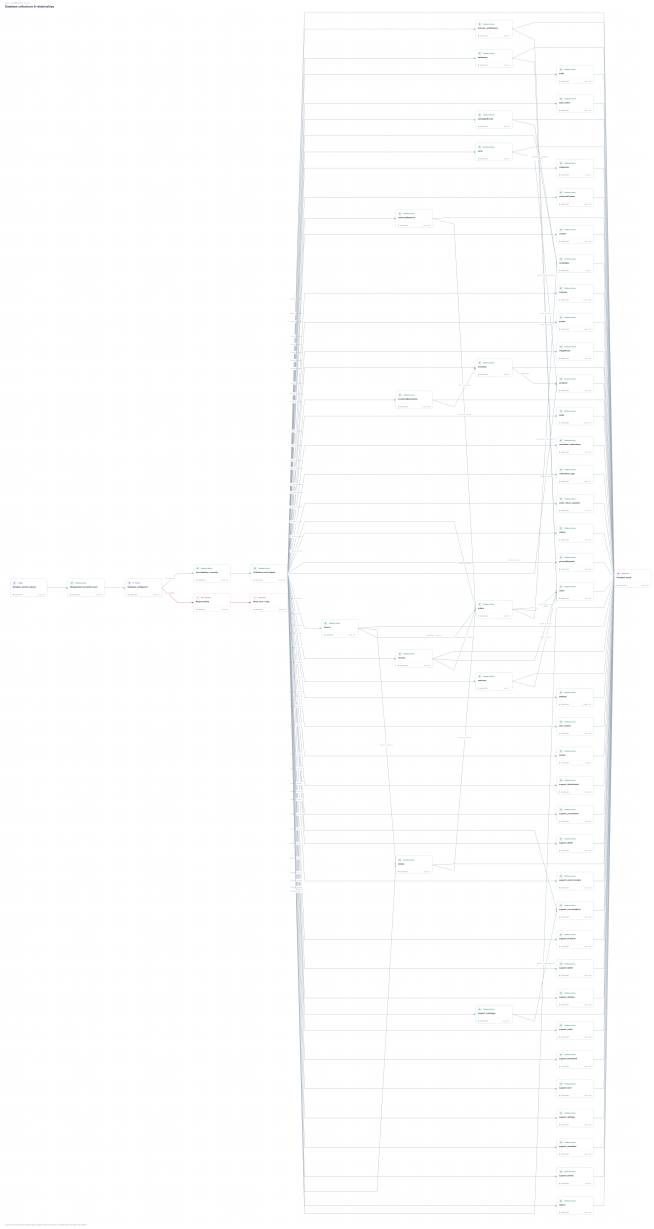
#03 Handler-specific guards -> #24 Other APIs | Next

#24 Other APIs -> #04 Database / service boundary | Next

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Database collections & relationships

Architecture overview. Readable node and connection sheets follow.



Database collections & relationships

Readable detail sheet 1/14. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Database service request

Backend requests a shared MongoDB client.

src/server/db.ts:30

Next -> #2 (p2)

#02 / Database Query / Implemented

MongoClient connection pool

MONGODB_URI environment lookup, pool of 10 and six-second selection timeout. No URI is displayed.

src/server/db.ts:34

Next -> #3 (p2)

#03 / IF / Switch / Implemented

Database configured?

Live services fail without MongoDB. A legacy local adapter exists after an unconditional early rejection and is not a working fallback.

src/server/db.ts:31

Yes / accepted -> #4 (p2); No / rejected -> #7 (p3)

#04 / Database Query / Implemented

ovia database / override

Configured DB name or ovia default; deployed Atlas topology is Unverified.

src/server/db.ts:45

Next -> #5 (p3)

Database collections & relationships

Readable detail sheet 2/14. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

Collections and indexes

setupDatabase defines identity, email uniqueness, token TTL and checkout idempotency indexes.

src/server/db.ts:239

Next -> #6 (p3); Collection access -> #9 (p4); Collection access -> #10 (p4); Collection access -> #11 (p4); Collection access -> #12 (p4); Collection access -> #13 (p5); Collection access -> #14 (p5); Collection access -> #15 (p5); Collection...

#06 / Response / Implemented

Persisted result

Repository reads and transactional writes return stored records.

src/server/db.ts:100

END: no outgoing connection

#07 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #8 (p3)

#08 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Database collections & relationships

Readable detail sheet 3/14. Node numbers and page references preserve every directed connection.

#09 / Database Query / Implemented

account_notifications

Actual collection name used by source. Evidence: 7 literal references. Production contents are Unverified.

src/app/api/v1/account/activity/route.ts:17

Next -> #6 (p3); userId -> users.id -> #56 (p15)

#10 / Database Query / Implemented

addresses

Actual collection name used by source. Evidence: 9 literal references. Production contents are Unverified.

src/app/api/v1/account/addresses/[[...id]]/route.ts:19

Next -> #6 (p3); userId -> users.id -> #56 (p15)

#11 / Database Query / Implemented

audit

Actual collection name used by source. Evidence: 13 literal references. Production contents are Unverified.

src/app/api/admin/support/route.ts:196

Next -> #6 (p3)

#12 / Database Query / Implemented

bulk_orders

Actual collection name used by source. Evidence: 3 literal references. Production contents are Unverified.

src/app/api/v1/bulk-orders/route.ts:142

Next -> #6 (p3)

Database collections & relationships

Readable detail sheet 4/14. Node numbers and page references preserve every directed connection.

#13 / Database Query / Implemented

campaignEvents

Actual collection name used by source. Evidence: 3 literal references. Production contents are Unverified.

src/server/campaigns.ts:20

Next -> #6 (p3); campaignId -> campaign identity -> #14 (p5)

#14 / Database Query / Implemented

campaigns

Actual collection name used by source. Evidence: 4 literal references. Production contents are Unverified.

src/server/campaigns.ts:11

Next -> #6 (p3)

#15 / Database Query / Implemented

carts

Actual collection name used by source. Evidence: 18 literal references. Production contents are Unverified.

src/server/account-sign-in.ts:7

Next -> #6 (p3); lines[].sku -> variants[].sku -> #32 (p9)

#16 / Database Query / Implemented

categories

Actual collection name used by source. Evidence: 5 literal references. Production contents are Unverified.

src/app/api/admin/product-taxonomy/route.ts:14

Next -> #6 (p3)

Database collections & relationships

Readable detail sheet 5/14. Node numbers and page references preserve every directed connection.

#17 / Database Query / Implemented

checkoutPromos

Actual collection name used by source. Evidence: 3 literal references. Production contents are Unverified.

src/server/checkout-pricing.ts:132

Next -> #6 (p3)

#18 / Database Query / Implemented

checkoutSessions

Actual collection name used by source. Evidence: 15 literal references. Production contents are Unverified.

src/server/commerce.ts:136

Next -> #6 (p3); orderId -> orders.id -> #29 (p9)

#19 / Database Query / Implemented

contact

Actual collection name used by source. Evidence: 4 literal references. Production contents are Unverified.

src/app/api/v1/[[...path]]/route.ts:450

Next -> #6 (p3)

#20 / Database Query / Implemented

coupons

Actual collection name used by source. Evidence: 18 literal references. Production contents are Unverified.

src/server/admin-service.ts:1546

Next -> #6 (p3)

Database collections & relationships

Readable detail sheet 6/14. Node numbers and page references preserve every directed connection.

#21 / Database Query / Implemented

events

Actual collection name used by source. Evidence: 20 literal references. Production contents are Unverified.

src/app/api/admin/content/blog-events/route.ts:127

Next -> #6 (p3)

#22 / Database Query / Implemented

integrations

Actual collection name used by source. Evidence: 6 literal references. Production contents are Unverified.

src/server/integrations-service.ts:53

Next -> #6 (p3)

#23 / Database Query / Implemented

inventory

Actual collection name used by source. Evidence: 17 literal references. Production contents are Unverified.

src/app/api/v1/orders/[id]/return/route.ts:77

Next -> #6 (p3); id -> variants[].sku -> #32 (p9)

#24 / Database Query / Implemented

inventoryMovements

Actual collection name used by source. Evidence: 2 literal references. Production contents are Unverified.

src/server/admin-service.ts:1206

Next -> #6 (p3); sku -> inventory.id -> #23 (p7)

Database collections & relationships

Readable detail sheet 7/14. Node numbers and page references preserve every directed connection.

#25 / Database Query / Implemented

limits

Actual collection name used by source. Evidence: 5 literal references. Production contents are Unverified.

src/server/http.ts:114

Next -> #6 (p3)

#26 / Database Query / Implemented

newsletter_subscribers

Actual collection name used by source. Evidence: 1 literal references. Production contents are Unverified.

src/app/api/v1/account/newsletter/route.ts:22

Next -> #6 (p3)

#27 / Database Query / Implemented

notification_logs

Actual collection name used by source. Evidence: 6 literal references. Production contents are Unverified.

src/server/notification-service.ts:751

Next -> #6 (p3)

#28 / Database Query / Implemented

order_return_requests

Actual collection name used by source. Evidence: 3 literal references. Production contents are Unverified.

src/app/api/admin/orders/[id]/return-request/route.ts:17

Next -> #6 (p3)

Database collections & relationships

Readable detail sheet 8/14. Node numbers and page references preserve every directed connection.

#29 / Database Query / Implemented

orders

Actual collection name used by source. Evidence: 112 literal references. Production contents are Unverified.

src/app/api/admin/orders/[id]/document/route.ts:16

Next -> #6 (p3); userId / owner -> user identity -> #56 (p15); campaignId -> campaigns.id -> #14 (p5)

#30 / Database Query / Implemented

outbox

Actual collection name used by source. Evidence: 30 literal references. Production contents are Unverified.

src/app/api/admin/shipping/route.ts:63

Next -> #6 (p3)

#31 / Database Query / Implemented

privacyRequests

Actual collection name used by source. Evidence: 8 literal references. Production contents are Unverified.

src/app/api/v1/care/[[...path]]/route.ts:58

Next -> #6 (p3)

#32 / Database Query / Implemented

products

Actual collection name used by source. Evidence: 20 literal references. Production contents are Unverified.

src/app/api/admin/product-taxonomy/route.ts:30

Next -> #6 (p3)

Database collections & relationships

Readable detail sheet 9/14. Node numbers and page references preserve every directed connection.

#33 / Database Query / Implemented

returns

Actual collection name used by source. Evidence: 2 literal references. Production contents are Unverified.

src/server/admin-service.ts:1401

Next -> #6 (p3); ticketId -> tickets.id -> #54 (p15); orderRawId -> orders.id -> #29 (p9)

#34 / Database Query / Implemented

reviews

Actual collection name used by source. Evidence: 9 literal references. Production contents are Unverified.

src/app/api/v1/reviews/route.ts:63

Next -> #6 (p3); userId -> users.id -> #56 (p15); optional orderId -> orders.id -> #29 (p9)

#35 / Database Query / Implemented

sessions

Actual collection name used by source. Evidence: 24 literal references. Production contents are Unverified.

src/app/api/admin/security/route.ts:17

Next -> #6 (p3); userId -> users.id -> #56 (p15)

#36 / Database Query / Implemented

settings

Actual collection name used by source. Evidence: 4 literal references. Production contents are Unverified.

src/server/admin-service.ts:1818

Next -> #6 (p3)

Database collections & relationships

Readable detail sheet 10/14. Node numbers and page references preserve every directed connection.

#37 / Database Query / Implemented

site_content

Actual collection name used by source. Evidence: 4 literal references. Production contents are Unverified.

src/app/api/admin/content/route.ts:230

Next -> #6 (p3)

#38 / Database Query / Implemented

stories

Actual collection name used by source. Evidence: 16 literal references. Production contents are Unverified.

src/app/api/admin/articles/route.ts:11

Next -> #6 (p3)

#39 / Database Query / Implemented

support_attachments

Actual collection name used by source. Evidence: 6 literal references. Production contents are Unverified.

src/app/api/support/attachments/[id]/route.ts:18

Next -> #6 (p3)

#40 / Database Query / Implemented

support_connections

Actual collection name used by source. Evidence: 5 literal references. Production contents are Unverified.

src/app/api/admin/support/gmail/callback/route.ts:58

Next -> #6 (p3)

Database collections & relationships

Readable detail sheet 11/14. Node numbers and page references preserve every directed connection.

#41 / Database Query / Implemented

support_conversations

Actual collection name used by source. Evidence: 17 literal references. Production contents are Unverified.

src/app/api/v1/care/[[...path]]/route.ts:453

Next -> #6 (p3)

#42 / Database Query / Implemented

support_drafts

Actual collection name used by source. Evidence: 4 literal references. Production contents are Unverified.

src/server/support-mailbox.ts:283

Next -> #6 (p3)

#43 / Database Query / Implemented

support_email_receipts

Actual collection name used by source. Evidence: 7 literal references. Production contents are Unverified.

src/server/support-gmail.ts:153

Next -> #6 (p3)

#44 / Database Query / Implemented

support_forwards

Actual collection name used by source. Evidence: 3 literal references. Production contents are Unverified.

src/server/support-mailbox.ts:442

Next -> #6 (p3)

Database collections & relationships

Readable detail sheet 12/14. Node numbers and page references preserve every directed connection.

#45 / Database Query / Implemented

support_labels

Actual collection name used by source. Evidence: 3 literal references. Production contents are Unverified.

src/server/support-mailbox.ts:343

Next -> #6 (p3)

#46 / Database Query / Implemented

support_mailbox

Actual collection name used by source. Evidence: 8 literal references. Production contents are Unverified.

src/server/support-inbox.ts:363

Next -> #6 (p3)

#47 / Database Query / Implemented

support_messages

Actual collection name used by source. Evidence: 7 literal references. Production contents are Unverified.

src/app/api/v1/care/[[...path]]/route.ts:231

Next -> #6 (p3); ticketId -> conversation ID -> #41 (p12)

#48 / Database Query / Implemented

support_notes

Actual collection name used by source. Evidence: 5 literal references. Production contents are Unverified.

src/server/support-conversations.ts:198

Next -> #6 (p3)

Database collections & relationships

Readable detail sheet 13/14. Node numbers and page references preserve every directed connection.

#49 / Database Query / Implemented

support_scheduled

Actual collection name used by source. Evidence: 6 literal references. Production contents are Unverified.

src/server/support-mailbox.ts:305

Next -> #6 (p3)

#50 / Database Query / Implemented

support_sent

Actual collection name used by source. Evidence: 4 literal references. Production contents are Unverified.

src/server/support-mailbox.ts:403

Next -> #6 (p3)

#51 / Database Query / Implemented

support_settings

Actual collection name used by source. Evidence: 2 literal references. Production contents are Unverified.

src/server/support-mailbox.ts:367

Next -> #6 (p3)

#52 / Database Query / Implemented

support_templates

Actual collection name used by source. Evidence: 3 literal references. Production contents are Unverified.

src/server/support-mailbox.ts:323

Next -> #6 (p3)

Database collections & relationships

Readable detail sheet 14/14. Node numbers and page references preserve every directed connection.

#53 / Database Query / Implemented

support_tickets

Actual collection name used by source. Evidence: 7 literal references. Production contents are Unverified.

src/app/api/v1/support/route.ts:75

Next -> #6 (p3)

#54 / Database Query / Implemented

tickets

Actual collection name used by source. Evidence: 18 literal references. Production contents are Unverified.

src/app/api/v1/account/activity/route.ts:14

Next -> #6 (p3); orderId -> orders.id -> #29 (p9)

#55 / Database Query / Implemented

tokens

Actual collection name used by source. Evidence: 29 literal references. Production contents are Unverified.

src/server/db.ts:270

Next -> #6 (p3)

#56 / Database Query / Implemented

users

Actual collection name used by source. Evidence: 53 literal references. Production contents are Unverified.

src/app/api/admin/customers/route.ts:59

Next -> #6 (p3)

Database collections & relationships

Complete directed connection register / no omitted success, failure or proposed edges

#01 Database service request -> #02 MongoClient connection pool | Next

#02 MongoClient connection pool -> #03 Database configured? | Next

#03 Database configured? -> #04 ovia database / override | Yes / accepted

#04 ovia database / override -> #05 Collections and indexes | Next

#05 Collections and indexes -> #06 Persisted result | Next

#03 Database configured? -> #07 Request failed | No / rejected | FAILURE

#07 Request failed -> #08 Show error / retry | Next | FAILURE

#05 Collections and indexes -> #09 account_notifications | Collection access

#09 account_notifications -> #06 Persisted result | Next

#05 Collections and indexes -> #10 addresses | Collection access

#10 addresses -> #06 Persisted result | Next

#05 Collections and indexes -> #11 audit | Collection access

#11 audit -> #06 Persisted result | Next

#05 Collections and indexes -> #12 bulk_orders | Collection access

#12 bulk_orders -> #06 Persisted result | Next

#05 Collections and indexes -> #13 campaignEvents | Collection access

#13 campaignEvents -> #06 Persisted result | Next

#05 Collections and indexes -> #14 campaigns | Collection access

#14 campaigns -> #06 Persisted result | Next

#05 Collections and indexes -> #15 carts | Collection access

#15 carts -> #06 Persisted result | Next

#05 Collections and indexes -> #16 categories | Collection access

Database collections & relationships

Directed connection register continued

#16 categories -> #06 Persisted result | Next

#05 Collections and indexes -> #17 checkoutPromos | Collection access

#17 checkoutPromos -> #06 Persisted result | Next

#05 Collections and indexes -> #18 checkoutSessions | Collection access

#18 checkoutSessions -> #06 Persisted result | Next

#05 Collections and indexes -> #19 contact | Collection access

#19 contact -> #06 Persisted result | Next

#05 Collections and indexes -> #20 coupons | Collection access

#20 coupons -> #06 Persisted result | Next

#05 Collections and indexes -> #21 events | Collection access

#21 events -> #06 Persisted result | Next

#05 Collections and indexes -> #22 integrations | Collection access

#22 integrations -> #06 Persisted result | Next

#05 Collections and indexes -> #23 inventory | Collection access

#23 inventory -> #06 Persisted result | Next

#05 Collections and indexes -> #24 inventoryMovements | Collection access

#24 inventoryMovements -> #06 Persisted result | Next

#05 Collections and indexes -> #25 limits | Collection access

#25 limits -> #06 Persisted result | Next

#05 Collections and indexes -> #26 newsletter_subscribers | Collection access

#26 newsletter_subscribers -> #06 Persisted result | Next

#05 Collections and indexes -> #27 notification_logs | Collection access

Database collections & relationships

Directed connection register continued

#27 notification_logs -> #06 Persisted result | Next

#05 Collections and indexes -> #28 order_return_requests | Collection access

#28 order_return_requests -> #06 Persisted result | Next

#05 Collections and indexes -> #29 orders | Collection access

#29 orders -> #06 Persisted result | Next

#05 Collections and indexes -> #30 outbox | Collection access

#30 outbox -> #06 Persisted result | Next

#05 Collections and indexes -> #31 privacyRequests | Collection access

#31 privacyRequests -> #06 Persisted result | Next

#05 Collections and indexes -> #32 products | Collection access

#32 products -> #06 Persisted result | Next

#05 Collections and indexes -> #33 returns | Collection access

#33 returns -> #06 Persisted result | Next

#05 Collections and indexes -> #34 reviews | Collection access

#34 reviews -> #06 Persisted result | Next

#05 Collections and indexes -> #35 sessions | Collection access

#35 sessions -> #06 Persisted result | Next

#05 Collections and indexes -> #36 settings | Collection access

#36 settings -> #06 Persisted result | Next

#05 Collections and indexes -> #37 site_content | Collection access

#37 site_content -> #06 Persisted result | Next

#05 Collections and indexes -> #38 stories | Collection access

Database collections & relationships

Directed connection register continued

#38 stories -> #06 Persisted result | Next

#05 Collections and indexes -> #39 support_attachments | Collection access

#39 support_attachments -> #06 Persisted result | Next

#05 Collections and indexes -> #40 support_connections | Collection access

#40 support_connections -> #06 Persisted result | Next

#05 Collections and indexes -> #41 support_conversations | Collection access

#41 support_conversations -> #06 Persisted result | Next

#05 Collections and indexes -> #42 support_drafts | Collection access

#42 support_drafts -> #06 Persisted result | Next

#05 Collections and indexes -> #43 support_email_receipts | Collection access

#43 support_email_receipts -> #06 Persisted result | Next

#05 Collections and indexes -> #44 support_forwards | Collection access

#44 support_forwards -> #06 Persisted result | Next

#05 Collections and indexes -> #45 support_labels | Collection access

#45 support_labels -> #06 Persisted result | Next

#05 Collections and indexes -> #46 support_mailbox | Collection access

#46 support_mailbox -> #06 Persisted result | Next

#05 Collections and indexes -> #47 support_messages | Collection access

#47 support_messages -> #06 Persisted result | Next

#05 Collections and indexes -> #48 support_notes | Collection access

#48 support_notes -> #06 Persisted result | Next

#05 Collections and indexes -> #49 support_scheduled | Collection access

Database collections & relationships

Directed connection register continued

#49 support_scheduled -> #06 Persisted result | Next

#05 Collections and indexes -> #50 support_sent | Collection access

#50 support_sent -> #06 Persisted result | Next

#05 Collections and indexes -> #51 support_settings | Collection access

#51 support_settings -> #06 Persisted result | Next

#05 Collections and indexes -> #52 support_templates | Collection access

#52 support_templates -> #06 Persisted result | Next

#05 Collections and indexes -> #53 support_tickets | Collection access

#53 support_tickets -> #06 Persisted result | Next

#05 Collections and indexes -> #54 tickets | Collection access

#54 tickets -> #06 Persisted result | Next

#05 Collections and indexes -> #55 tokens | Collection access

#55 tokens -> #06 Persisted result | Next

#05 Collections and indexes -> #56 users | Collection access

#56 users -> #06 Persisted result | Next

#35 sessions -> #56 users | userId -> users.id

#10 addresses -> #56 users | userId -> users.id

#29 orders -> #56 users | userId / owner -> user identity

#18 checkoutSessions -> #29 orders | orderId -> orders.id

#23 inventory -> #32 products | id -> variants[].sku

#15 carts -> #32 products | lines[].sku -> variants[].sku

#34 reviews -> #56 users | userId -> users.id

Database collections & relationships

Directed connection register continued

#34 reviews -> #29 orders | optional orderId -> orders.id

#54 tickets -> #29 orders | orderId -> orders.id

#33 returns -> #54 tickets | ticketId -> tickets.id

#33 returns -> #29 orders | orderRawId -> orders.id

#29 orders -> #14 campaigns | campaignId -> campaigns.id

#13 campaignEvents -> #14 campaigns | campaignId -> campaign identity

#47 support_messages -> #41 support_conversations | ticketId -> conversation ID

#09 account_notifications -> #56 users | userId -> users.id

#24 inventoryMovements -> #23 inventory | sku -> inventory.id

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

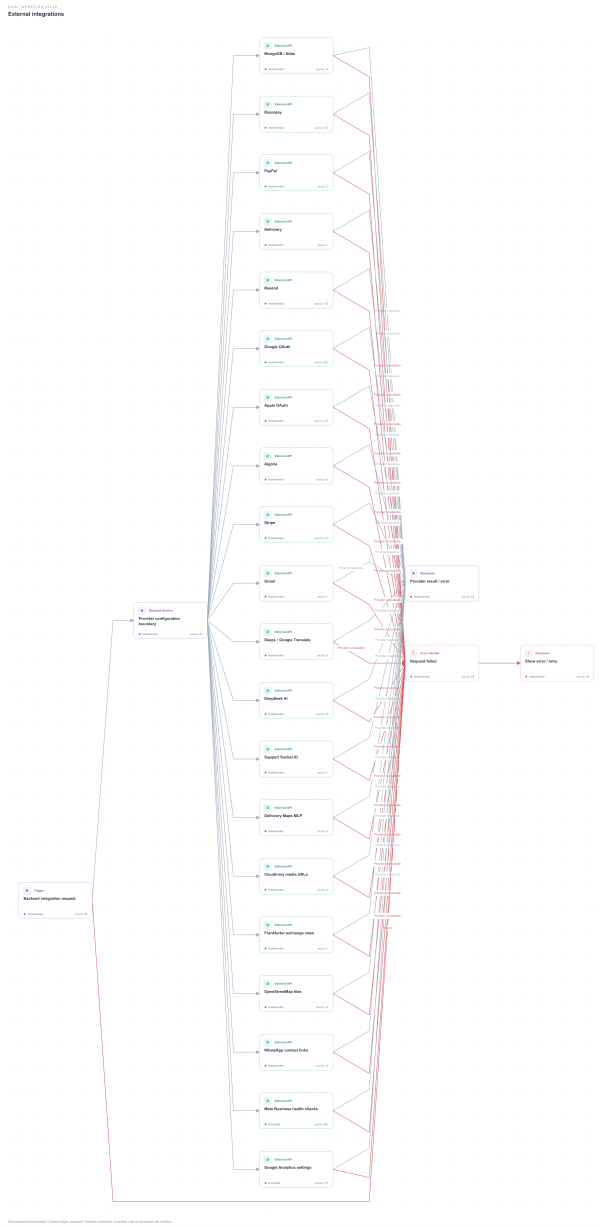
Note: 48 literal collection names were found in queries or index definitions. A collection name in code does not prove that the collection exists in production.

Note: Payments, refunds and shipments are embedded in orders. Wishlist is embedded in users. Categories are matched by catalog fields; no category foreign key is declared.

Note: No Mongoose schemas were found; data models are TypeScript types plus runtime route validation.

External integrations

Architecture overview. Readable node and connection sheets follow.



External integrations

Readable detail sheet 1/7. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Backend integration request

Service code selects a provider for the requested task.

src/server/integrations-service.ts:88

Next -> #2 (p2); Failure -> #4 (p2)

#02 / Backend Service / Implemented

Provider configuration boundary

Read environment and stored integration settings; no credential values appear in the editor.

src/server/integrations-service.ts:50

Next -> #6 (p3); Next -> #7 (p3); Next -> #8 (p3); Next -> #9 (p4); Next -> #10 (p4); Next -> #11 (p4); Next -> #12 (p4); Next -> #13 (p5); Next -> #14 (p5); Next -> #15 (p5); Next -> #16 (p5); Next -> #17 (p6); Next -> #18 (p6); Next -> #19 (p6); Next...

#03 / Response / Implemented

Provider result / error

Caller receives provider-confirmed data or a route-specific service error.

src/server/http.ts:69

END: no outgoing connection

#04 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #5 (p3)

External integrations

Readable detail sheet 2/7. Node numbers and page references preserve every directed connection.

#05 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

#06 / External API / Implemented

MongoDB / Atlas

Native driver and transactional repository. Hosting on Atlas, cluster layout and live connectivity are Unverified.

src/server/db.ts:30

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#07 / External API / Implemented

Razorpay

Orders, captured payments, signed webhooks and refunds. Credentials, enabled webhook events and merchant connectivity are Unverified.

src/server/razorpay-checkout.ts:322

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#08 / External API / Implemented

PayPal

Sandbox orders, approval, capture and verified webhook. Sandbox endpoint is explicit. Live PayPal settlement and automatic PayPal refunds are Planned.

src/server/paypal.ts:3

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

External integrations

Readable detail sheet 3/7. Node numbers and page references preserve every directed connection.

#09 / External API / Implemented

Delhivery

Serviceability, shipment, pickup, cancellation and tracking. Courier configuration and live operational readiness are Unverified.

src/lib/shipping/delhivery.ts:1

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#10 / External API / Implemented

Resend

Outbound account, order, support and notification email. Sender domain, API configuration and delivered email are Unverified.

src/server/mail.ts:193

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#11 / External API / Implemented

Google OAuth

State / PKCE and JWKS-verified identity. Enabled provider configuration is Unverified.

src/server/social-auth.ts:261

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#12 / External API / Implemented

Apple OAuth

Identity verification and client-secret signing. Apple keys, enabled provider and runtime readiness are Unverified.

src/server/social-auth.ts:232

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

External integrations

Readable detail sheet 4/7. Node numbers and page references preserve every directed connection.

#13 / External API / Implemented

Algolia

Search SDK and optional product index synchronization. Index freshness and configured keys are Unverified.

src/lib/algolia.ts:42

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#14 / External API / Implemented

Stripe

Legacy test-mode checkout and signed webhook. Checkout is guarded to sk_test_ credentials; live checkout is not claimed.

src/app/api/v1/[[...path]]/route.ts:413

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#15 / External API / Implemented

Gmail

OAuth mailbox connection and sync. Mailbox connection, grants and scheduler execution are Unverified.

src/server/support-gmail.ts:1

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#16 / External API / Implemented

DeepL / Google Translate

HTTP translation provider selection. Provider selection depends on environment; active configuration is Unverified.

src/app/api/translate/route.ts:8

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

External integrations

Readable detail sheet 5/7. Node numbers and page references preserve every directed connection.

#17 / External API / Implemented

DeepSeek AI

Owner copilot with constrained provider endpoint. Provider connectivity and model response quality are Unverified.

src/server/ai-copilot.ts:25

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#18 / External API / Implemented

Support Socket.IO

Separate realtime support transport. Local support process / deployment transport availability is Unverified.

src/components/use-support-realtime.ts:1

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#19 / External API / Implemented

Delhivery Maps MCP

JSON-RPC tools/call for address, search and route tools. Active token and tool availability are Unverified. A hardcoded fallback token exists in source; its value is excluded from documentation.

src/lib/shipping/delhivery-maps.ts:9

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#20 / External API / Implemented

Cloudinary media URLs

Validated public-ID image transformation URL helper. The helper creates CDN URLs. Admin uploads currently write to local public/media/uploads; Cloudinary upload API implementation is not claimed.

src/lib/media.ts:2

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

External integrations

Readable detail sheet 6/7. Node numbers and page references preserve every directed connection.

#21 / External API / Implemented

Frankfurter exchange rates

INR display-rate HTTP fetch with cache. Rates are approximate display conversions; provider failure returns an empty rate list. Runtime availability is Unverified.

src/app/api/v1/rates/route.ts:7

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#22 / External API / Implemented

OpenStreetMap tiles

Leaflet browser tile requests for store maps. Public tile rendering is implemented; availability and usage at runtime are Unverified.

src/components/store-map.tsx:20

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#23 / External API / Implemented

WhatsApp contact links

Browser deep links to WhatsApp / wa.me. This contact UI opens WhatsApp. It does not prove a server-side automated message-sending API exists.

src/components/whatsapp-widget.tsx:41

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

#24 / External API / Unverified

Meta Business health checks

Stored settings and Graph API identity checks. Credential-check code exists for WhatsApp/Messenger. Automated message delivery and a verified Messenger webhook are not established by these checks.

src/server/integrations-service.ts:662

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

External integrations

Readable detail sheet 7/7. Node numbers and page references preserve every directed connection.

#25 / External API / Unverified

Google Analytics settings

Measurement-ID storage and format check. Settings UI exists. No gtag/GA collection call was found in src; automatic GA4 event delivery is Planned, not proven by ID validation.

src/server/integrations-service.ts:711

Provider response -> #3 (p2); Provider unavailable -> #4 (p2)

External integrations

Complete directed connection register / no omitted success, failure or proposed edges

#01 Backend integration request -> #02 Provider configuration boundary | Next

#01 Backend integration request -> #04 Request failed | Failure | FAILURE

#04 Request failed -> #05 Show error / retry | Next | FAILURE

#02 Provider configuration boundary -> #06 MongoDB / Atlas | Next

#06 MongoDB / Atlas -> #03 Provider result / error | Provider response

#06 MongoDB / Atlas -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #07 Razorpay | Next

#07 Razorpay -> #03 Provider result / error | Provider response

#07 Razorpay -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #08 PayPal | Next

#08 PayPal -> #03 Provider result / error | Provider response

#08 PayPal -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #09 Delhivery | Next

#09 Delhivery -> #03 Provider result / error | Provider response

#09 Delhivery -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #10 Resend | Next

#10 Resend -> #03 Provider result / error | Provider response

#10 Resend -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #11 Google OAuth | Next

#11 Google OAuth -> #03 Provider result / error | Provider response

#11 Google OAuth -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #12 Apple OAuth | Next

External integrations

Directed connection register continued

#12 Apple OAuth -> #03 Provider result / error | Provider response

#12 Apple OAuth -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #13 Algolia | Next

#13 Algolia -> #03 Provider result / error | Provider response

#13 Algolia -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #14 Stripe | Next

#14 Stripe -> #03 Provider result / error | Provider response

#14 Stripe -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #15 Gmail | Next

#15 Gmail -> #03 Provider result / error | Provider response

#15 Gmail -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #16 DeepL / Google Translate | Next

#16 DeepL / Google Translate -> #03 Provider result / error | Provider response

#16 DeepL / Google Translate -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #17 DeepSeek AI | Next

#17 DeepSeek AI -> #03 Provider result / error | Provider response

#17 DeepSeek AI -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #18 Support Socket.IO | Next

#18 Support Socket.IO -> #03 Provider result / error | Provider response

#18 Support Socket.IO -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #19 Delhivery Maps MCP | Next

#19 Delhivery Maps MCP -> #03 Provider result / error | Provider response

External integrations

Directed connection register continued

#19 Delhivery Maps MCP -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #20 Cloudinary media URLs | Next

#20 Cloudinary media URLs -> #03 Provider result / error | Provider response

#20 Cloudinary media URLs -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #21 Frankfurter exchange rates | Next

#21 Frankfurter exchange rates -> #03 Provider result / error | Provider response

#21 Frankfurter exchange rates -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #22 OpenStreetMap tiles | Next

#22 OpenStreetMap tiles -> #03 Provider result / error | Provider response

#22 OpenStreetMap tiles -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #23 WhatsApp contact links | Next

#23 WhatsApp contact links -> #03 Provider result / error | Provider response

#23 WhatsApp contact links -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #24 Meta Business health checks | Next

#24 Meta Business health checks -> #03 Provider result / error | Provider response

#24 Meta Business health checks -> #04 Request failed | Provider unavailable | FAILURE

#02 Provider configuration boundary -> #25 Google Analytics settings | Next

#25 Google Analytics settings -> #03 Provider result / error | Provider response

#25 Google Analytics settings -> #04 Request failed | Provider unavailable | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: All listed adapters have code evidence. Runtime configuration is Unverified because this task does not read production secrets or call live providers. No integration is labeled Configured without configuration evidence.

User & account data

Architecture overview. Readable node and connection sheets follow.



User & account data

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Domain persistence request

Application services query the collections listed in this domain.

src/server/db.ts:44

Next -> #2 (p2); Failure -> #8 (p3)

#02 / Database Query / Implemented

users

Code-backed collection users; view reference fields in the model inventory.

src/app/api/admin/customers/route.ts:59

Next -> #3 (p2)

#03 / Database Query / Implemented

sessions

Code-backed collection sessions; view reference fields in the model inventory.

src/app/api/admin/security/route.ts:17

Next -> #4 (p2)

#04 / Database Query / Implemented

addresses

Code-backed collection addresses; view reference fields in the model inventory.

src/app/api/v1/account/addresses/[[...id]]/route.ts:19

Next -> #5 (p3)

User & account data

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

tokens

Code-backed collection tokens; view reference fields in the model inventory.

src/server/db.ts:270

Next -> #6 (p3)

#06 / Database Query / Implemented

account_notifications

Code-backed collection account_notifications; view reference fields in the model inventory.

src/app/api/v1/account/activity/route.ts:17

Next -> #7 (p3)

#07 / Response / Implemented

Stored domain response

Return persisted domain data to the service.

src/server/db.ts:100

END: no outgoing connection

#08 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #9 (p4)

User & account data

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

User & account data

Complete directed connection register / no omitted success, failure or proposed edges

#01 Domain persistence request -> #02 users | Next

#02 users -> #03 sessions | Next

#03 sessions -> #04 addresses | Next

#04 addresses -> #05 tokens | Next

#05 tokens -> #06 account_notifications | Next

#06 account_notifications -> #07 Stored domain response | Next

#01 Domain persistence request -> #08 Request failed | Failure | FAILURE

#08 Request failed -> #09 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: sessions.userId -> users.id (src/server/auth.ts:128)

Note: addresses.userId -> users.id (src/app/api/v1/account/addresses/[[...id]]/route.ts:8)

Note: orders.userId / owner -> user identity (src/server/commerce.ts:171)

Note: reviews.userId -> users.id (src/app/api/v1/reviews/route.ts:63)

Note: account_notifications.userId -> users.id (src/server/account-activity.ts:9)

Catalog & stock data

Architecture overview. Readable node and connection sheets follow.



Catalog & stock data

Readable detail sheet 1/2. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Domain persistence request

Application services query the collections listed in this domain.

src/server/db.ts:44

Next -> #2 (p2); Failure -> #7 (p3)

#02 / Database Query / Implemented

products

Code-backed collection products; view reference fields in the model inventory.

src/app/api/admin/product-taxonomy/route.ts:30

Next -> #3 (p2)

#03 / Database Query / Implemented

categories

Code-backed collection categories; view reference fields in the model inventory.

src/app/api/admin/product-taxonomy/route.ts:14

Next -> #4 (p2)

#04 / Database Query / Implemented

inventory

Code-backed collection inventory; view reference fields in the model inventory.

src/app/api/v1/orders/[id]/return/route.ts:77

Next -> #5 (p3)

Catalog & stock data

Readable detail sheet 2/2. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

inventoryMovements

Code-backed collection inventoryMovements; view reference fields in the model inventory.

src/server/admin-service.ts:1206

Next -> #6 (p3)

#06 / Response / Implemented

Stored domain response

Return persisted domain data to the service.

src/server/db.ts:100

END: no outgoing connection

#07 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #8 (p3)

#08 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Catalog & stock data

Complete directed connection register / no omitted success, failure or proposed edges

#01 Domain persistence request -> #02 products | Next

#02 products -> #03 categories | Next

#03 categories -> #04 inventory | Next

#04 inventory -> #05 inventoryMovements | Next

#05 inventoryMovements -> #06 Stored domain response | Next

#01 Domain persistence request -> #07 Request failed | Failure | FAILURE

#07 Request failed -> #08 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: inventory.id -> variants[].sku (src/server/db.ts:210)

Note: carts.lines[].sku -> variants[].sku (src/server/commerce.ts:25)

Note: inventoryMovements.sku -> inventory.id (src/server/admin-service.ts:1236)

Cart & checkout data

Architecture overview. Readable node and connection sheets follow.



Cart & checkout data

Readable detail sheet 1/2. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Domain persistence request

Application services query the collections listed in this domain.

src/server/db.ts:44

Next -> #2 (p2); Failure -> #7 (p3)

#02 / Database Query / Implemented

carts

Code-backed collection carts; view reference fields in the model inventory.

src/server/account-sign-in.ts:7

Next -> #3 (p2)

#03 / Database Query / Implemented

checkoutSessions

Code-backed collection checkoutSessions; view reference fields in the model inventory.

src/server/commerce.ts:136

Next -> #4 (p2)

#04 / Database Query / Implemented

checkoutPromos

Code-backed collection checkoutPromos; view reference fields in the model inventory.

src/server/checkout-pricing.ts:132

Next -> #5 (p3)

Cart & checkout data

Readable detail sheet 2/2. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

coupons

Code-backed collection coupons; view reference fields in the model inventory.

src/server/admin-service.ts:1546

Next -> #6 (p3)

#06 / Response / Implemented

Stored domain response

Return persisted domain data to the service.

src/server/db.ts:100

END: no outgoing connection

#07 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #8 (p3)

#08 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Cart & checkout data

Complete directed connection register / no omitted success, failure or proposed edges

#01 Domain persistence request -> #02 carts | Next

#02 carts -> #03 checkoutSessions | Next

#03 checkoutSessions -> #04 checkoutPromos | Next

#04 checkoutPromos -> #05 coupons | Next

#05 coupons -> #06 Stored domain response | Next

#01 Domain persistence request -> #07 Request failed | Failure | FAILURE

#07 Request failed -> #08 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: checkoutSessions.orderId -> orders.id (src/server/commerce.ts:191)

Note: carts.lines[].sku -> variants[].sku (src/server/commerce.ts:25)

Orders, payments & shipments

Architecture overview. Readable node and connection sheets follow.



Orders, payments & shipments

Readable detail sheet 1/2. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Domain persistence request

Application services query the collections listed in this domain.

src/server/db.ts:44

Next -> #2 (p2); Failure -> #6 (p3)

#02 / Database Query / Implemented

orders

Code-backed collection orders; view reference fields in the model inventory.

src/app/api/admin/orders/[id]/document/route.ts:16

Next -> #3 (p2)

#03 / Database Query / Implemented

events

Code-backed collection events; view reference fields in the model inventory.

src/app/api/admin/content/blog-events/route.ts:127

Next -> #4 (p2)

#04 / Database Query / Implemented

order_return_requests

Code-backed collection order_return_requests; view reference fields in the model inventory.

src/app/api/admin/orders/[id]/return-request/route.ts:17

Next -> #5 (p3)

Orders, payments & shipments

Readable detail sheet 2/2. Node numbers and page references preserve every directed connection.

#05 / Response / Implemented

Stored domain response

Return persisted domain data to the service.

src/server/db.ts:100

END: no outgoing connection

#06 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #7 (p3)

#07 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Orders, payments & shipments

Complete directed connection register / no omitted success, failure or proposed edges

#01 Domain persistence request -> #02 orders | Next

#02 orders -> #03 events | Next

#03 events -> #04 order_return_requests | Next

#04 order_return_requests -> #05 Stored domain response | Next

#01 Domain persistence request -> #06 Request failed | Failure | FAILURE

#06 Request failed -> #07 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: orders.userId / owner -> user identity (src/server/commerce.ts:171)

Note: checkoutSessions.orderId -> orders.id (src/server/commerce.ts:191)

Note: reviews.optional orderId -> orders.id (src/lib/types.ts:187)

Note: tickets.orderId -> orders.id (src/server/care-order-requests.ts:73)

Note: returns.orderRawId -> orders.id (src/server/admin-service.ts:1407)

Note: orders.campaignId -> campaigns.id (src/server/commerce.ts:178)

Note: PaymentInfo, providerId, captureId, refunds[], shipping and logisticsOperations are embedded on Order. No separate payment or shipment collection is asserted.

Returns & refund review data

Architecture overview. Readable node and connection sheets follow.



Returns & refund review data

Readable detail sheet 1/2. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Domain persistence request

Application services query the collections listed in this domain.

src/server/db.ts:44

Next -> #2 (p2); Failure -> #6 (p3)

#02 / Database Query / Implemented

tickets

Code-backed collection tickets; view reference fields in the model inventory.

src/app/api/v1/account/activity/route.ts:14

Next -> #3 (p2)

#03 / Database Query / Implemented

returns

Code-backed collection returns; view reference fields in the model inventory.

src/server/admin-service.ts:1401

Next -> #4 (p2)

#04 / Database Query / Implemented

orders

Code-backed collection orders; view reference fields in the model inventory.

src/app/api/admin/orders/[id]/document/route.ts:16

Next -> #5 (p3)

Returns & refund review data

Readable detail sheet 2/2. Node numbers and page references preserve every directed connection.

#05 / Response / Implemented

Stored domain response

Return persisted domain data to the service.

src/server/db.ts:100

END: no outgoing connection

#06 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #7 (p3)

#07 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Returns & refund review data

Complete directed connection register / no omitted success, failure or proposed edges

#01 Domain persistence request -> #02 tickets | Next

#02 tickets -> #03 returns | Next

#03 returns -> #04 orders | Next

#04 orders -> #05 Stored domain response | Next

#01 Domain persistence request -> #06 Request failed | Failure | FAILURE

#06 Request failed -> #07 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: orders.userId / owner -> user identity (src/server/commerce.ts:171)

Note: checkoutSessions.orderId -> orders.id (src/server/commerce.ts:191)

Note: reviews.optional orderId -> orders.id (src/lib/types.ts:187)

Note: tickets.orderId -> orders.id (src/server/care-order-requests.ts:73)

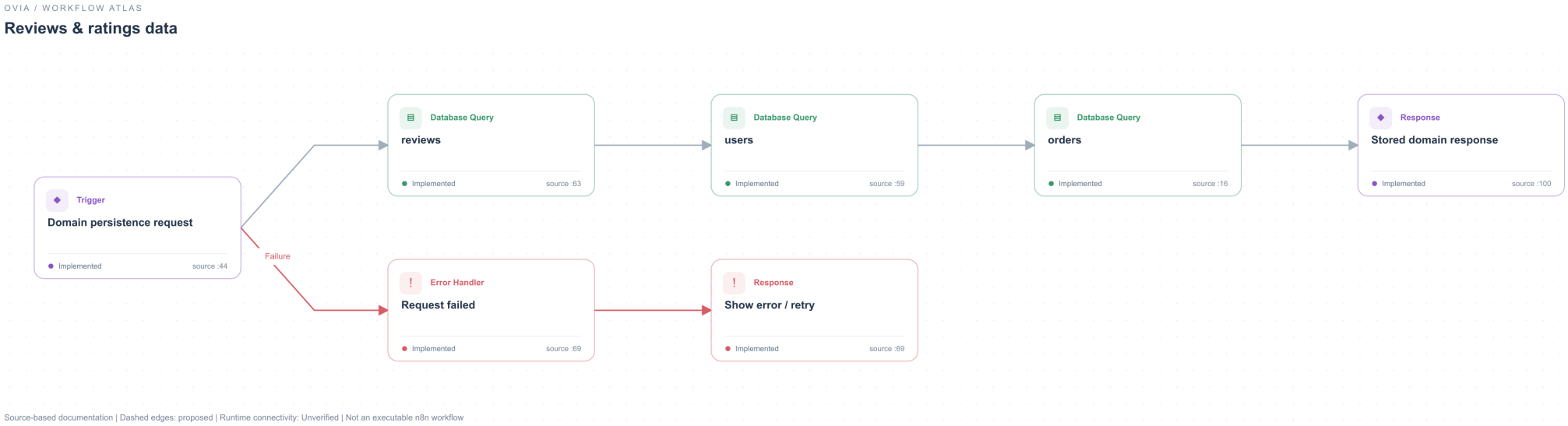
Note: returns.ticketId -> tickets.id (src/server/admin-service.ts:1407)

Note: returns.orderRawId -> orders.id (src/server/admin-service.ts:1407)

Note: orders.campaignId -> campaigns.id (src/server/commerce.ts:178)

Reviews & ratings data

Architecture overview. Readable node and connection sheets follow.



Reviews & ratings data

Readable detail sheet 1/2. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Domain persistence request

Application services query the collections listed in this domain.

src/server/db.ts:44

Next -> #2 (p2); Failure -> #6 (p3)

#02 / Database Query / Implemented

reviews

Code-backed collection reviews; view reference fields in the model inventory.

src/app/api/v1/reviews/route.ts:63

Next -> #3 (p2)

#03 / Database Query / Implemented

users

Code-backed collection users; view reference fields in the model inventory.

src/app/api/admin/customers/route.ts:59

Next -> #4 (p2)

#04 / Database Query / Implemented

orders

Code-backed collection orders; view reference fields in the model inventory.

src/app/api/admin/orders/[id]/document/route.ts:16

Next -> #5 (p3)

Reviews & ratings data

Readable detail sheet 2/2. Node numbers and page references preserve every directed connection.

#05 / Response / Implemented

Stored domain response

Return persisted domain data to the service.

src/server/db.ts:100

END: no outgoing connection

#06 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #7 (p3)

#07 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Reviews & ratings data

Complete directed connection register / no omitted success, failure or proposed edges

#01 Domain persistence request -> #02 reviews | Next

#02 reviews -> #03 users | Next

#03 users -> #04 orders | Next

#04 orders -> #05 Stored domain response | Next

#01 Domain persistence request -> #06 Request failed | Failure | FAILURE

#06 Request failed -> #07 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: sessions.userId -> users.id (src/server/auth.ts:128)

Note: addresses.userId -> users.id (src/app/api/v1/account/addresses/[[...id]]/route.ts:8)

Note: orders.userId / owner -> user identity (src/server/commerce.ts:171)

Note: checkoutSessions.orderId -> orders.id (src/server/commerce.ts:191)

Note: reviews.userId -> users.id (src/app/api/v1/reviews/route.ts:63)

Note: reviews.optional orderId -> orders.id (src/lib/types.ts:187)

Note: tickets.orderId -> orders.id (src/server/care-order-requests.ts:73)

Note: returns.orderRawId -> orders.id (src/server/admin-service.ts:1407)

Note: orders.campaignId -> campaigns.id (src/server/commerce.ts:178)

Note: account_notifications.userId -> users.id (src/server/account-activity.ts:9)

Support & conversation data

Architecture overview. Readable node and connection sheets follow.



Support & conversation data

Readable detail sheet 1/5. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Domain persistence request

Application services query the collections listed in this domain.

src/server/db.ts:44

Next -> #2 (p2); Failure -> #18 (p6)

#02 / Database Query / Implemented

support_attachments

Code-backed collection support_attachments; view reference fields in the model inventory.

src/app/api/support/attachments/[id]/route.ts:18

Next -> #3 (p2)

#03 / Database Query / Implemented

support_connections

Code-backed collection support_connections; view reference fields in the model inventory.

src/app/api/admin/support/gmail/callback/route.ts:58

Next -> #4 (p2)

#04 / Database Query / Implemented

support_conversations

Code-backed collection support_conversations; view reference fields in the model inventory.

src/app/api/v1/care/[[...path]]/route.ts:453

Next -> #5 (p3)

Support & conversation data

Readable detail sheet 2/5. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

support_drafts

Code-backed collection support_drafts; view reference fields in the model inventory.

src/server/support-mailbox.ts:283

Next -> #6 (p3)

#06 / Database Query / Implemented

support_email_receipts

Code-backed collection support_email_receipts; view reference fields in the model inventory.

src/server/support-gmail.ts:153

Next -> #7 (p3)

#07 / Database Query / Implemented

support_forwards

Code-backed collection support_forwards; view reference fields in the model inventory.

src/server/support-mailbox.ts:442

Next -> #8 (p3)

#08 / Database Query / Implemented

support_labels

Code-backed collection support_labels; view reference fields in the model inventory.

src/server/support-mailbox.ts:343

Next -> #9 (p4)

Support & conversation data

Readable detail sheet 3/5. Node numbers and page references preserve every directed connection.

#09 / Database Query / Implemented

support_mailbox

Code-backed collection support_mailbox; view reference fields in the model inventory.

src/server/support-inbox.ts:363

Next -> #10 (p4)

#10 / Database Query / Implemented

support_messages

Code-backed collection support_messages; view reference fields in the model inventory.

src/app/api/v1/care/[[...path]]/route.ts:231

Next -> #11 (p4)

#11 / Database Query / Implemented

support_notes

Code-backed collection support_notes; view reference fields in the model inventory.

src/server/support-conversations.ts:198

Next -> #12 (p4)

#12 / Database Query / Implemented

support_scheduled

Code-backed collection support_scheduled; view reference fields in the model inventory.

src/server/support-mailbox.ts:305

Next -> #13 (p5)

Support & conversation data

Readable detail sheet 4/5. Node numbers and page references preserve every directed connection.

#13 / Database Query / Implemented

support_sent

Code-backed collection support_sent; view reference fields in the model inventory.

src/server/support-mailbox.ts:403

Next -> #14 (p5)

#14 / Database Query / Implemented

support_settings

Code-backed collection support_settings; view reference fields in the model inventory.

src/server/support-mailbox.ts:367

Next -> #15 (p5)

#15 / Database Query / Implemented

support_templates

Code-backed collection support_templates; view reference fields in the model inventory.

src/server/support-mailbox.ts:323

Next -> #16 (p5)

#16 / Database Query / Implemented

support_tickets

Code-backed collection support_tickets; view reference fields in the model inventory.

src/app/api/v1/support/route.ts:75

Next -> #17 (p6)

Support & conversation data

Readable detail sheet 5/5. Node numbers and page references preserve every directed connection.

#17 / Response / Implemented

Stored domain response

Return persisted domain data to the service.

src/server/db.ts:100

END: no outgoing connection

#18 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #19 (p6)

#19 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Support & conversation data

Complete directed connection register / no omitted success, failure or proposed edges

#01 Domain persistence request -> #02 support_attachments | Next

#02 support_attachments -> #03 support_connections | Next

#03 support_connections -> #04 support_conversations | Next

#04 support_conversations -> #05 support_drafts | Next

#05 support_drafts -> #06 support_email_receipts | Next

#06 support_email_receipts -> #07 support_forwards | Next

#07 support_forwards -> #08 support_labels | Next

#08 support_labels -> #09 support_mailbox | Next

#09 support_mailbox -> #10 support_messages | Next

#10 support_messages -> #11 support_notes | Next

#11 support_notes -> #12 support_scheduled | Next

#12 support_scheduled -> #13 support_sent | Next

#13 support_sent -> #14 support_settings | Next

#14 support_settings -> #15 support_templates | Next

#15 support_templates -> #16 support_tickets | Next

#16 support_tickets -> #17 Stored domain response | Next

#01 Domain persistence request -> #18 Request failed | Failure | FAILURE

#18 Request failed -> #19 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: support_messages.ticketId -> conversation ID (src/server/support-conversations.ts:17)

Notification & mail data

Architecture overview. Readable node and connection sheets follow.



Notification & mail data

Readable detail sheet 1/2. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Domain persistence request

Application services query the collections listed in this domain.

src/server/db.ts:44

Next -> #2 (p2); Failure -> #6 (p3)

#02 / Database Query / Implemented

outbox

Code-backed collection outbox; view reference fields in the model inventory.

src/app/api/admin/shipping/route.ts:63

Next -> #3 (p2)

#03 / Database Query / Implemented

account_notifications

Code-backed collection account_notifications; view reference fields in the model inventory.

src/app/api/v1/account/activity/route.ts:17

Next -> #4 (p2)

#04 / Database Query / Implemented

notification_logs

Code-backed collection notification_logs; view reference fields in the model inventory.

src/server/notification-service.ts:751

Next -> #5 (p3)

Notification & mail data

Readable detail sheet 2/2. Node numbers and page references preserve every directed connection.

#05 / Response / Implemented

Stored domain response

Return persisted domain data to the service.

src/server/db.ts:100

END: no outgoing connection

#06 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #7 (p3)

#07 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Notification & mail data

Complete directed connection register / no omitted success, failure or proposed edges

#01 Domain persistence request -> #02 outbox | Next

#02 outbox -> #03 account_notifications | Next

#03 account_notifications -> #04 notification_logs | Next

#04 notification_logs -> #05 Stored domain response | Next

#01 Domain persistence request -> #06 Request failed | Failure | FAILURE

#06 Request failed -> #07 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: account_notifications.userId -> users.id (src/server/account-activity.ts:9)

Admin settings & audit data

Architecture overview. Readable node and connection sheets follow.



Admin settings & audit data

Readable detail sheet 1/2. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Domain persistence request

Application services query the collections listed in this domain.

src/server/db.ts:44

Next -> #2 (p2); Failure -> #7 (p3)

#02 / Database Query / Implemented

audit

Code-backed collection audit; view reference fields in the model inventory.

src/app/api/admin/support/route.ts:196

Next -> #3 (p2)

#03 / Database Query / Implemented

settings

Code-backed collection settings; view reference fields in the model inventory.

src/server/admin-service.ts:1818

Next -> #4 (p2)

#04 / Database Query / Implemented

integrations

Code-backed collection integrations; view reference fields in the model inventory.

src/server/integrations-service.ts:53

Next -> #5 (p3)

Admin settings & audit data

Readable detail sheet 2/2. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

site_content

Code-backed collection site_content; view reference fields in the model inventory.

src/app/api/admin/content/route.ts:230

Next -> #6 (p3)

#06 / Response / Implemented

Stored domain response

Return persisted domain data to the service.

src/server/db.ts:100

END: no outgoing connection

#07 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #8 (p3)

#08 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Admin settings & audit data

Complete directed connection register / no omitted success, failure or proposed edges

#01 Domain persistence request -> #02 audit | Next

#02 audit -> #03 settings | Next

#03 settings -> #04 integrations | Next

#04 integrations -> #05 site_content | Next

#05 site_content -> #06 Stored domain response | Next

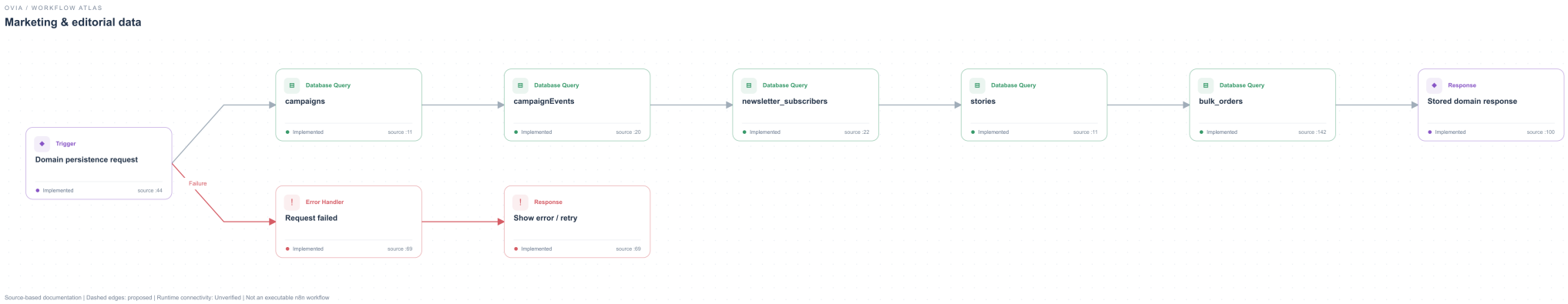
#01 Domain persistence request -> #07 Request failed | Failure | FAILURE

#07 Request failed -> #08 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Marketing & editorial data

Architecture overview. Readable node and connection sheets follow.



Marketing & editorial data

Readable detail sheet 1/3. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Domain persistence request

Application services query the collections listed in this domain.

src/server/db.ts:44

Next -> #2 (p2); Failure -> #8 (p3)

#02 / Database Query / Implemented

campaigns

Code-backed collection campaigns; view reference fields in the model inventory.

src/server/campaigns.ts:11

Next -> #3 (p2)

#03 / Database Query / Implemented

campaignEvents

Code-backed collection campaignEvents; view reference fields in the model inventory.

src/server/campaigns.ts:20

Next -> #4 (p2)

#04 / Database Query / Implemented

newsletter_subscribers

Code-backed collection newsletter_subscribers; view reference fields in the model inventory.

src/app/api/v1/account/newsletter/route.ts:22

Next -> #5 (p3)

Marketing & editorial data

Readable detail sheet 2/3. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

stories

Code-backed collection stories; view reference fields in the model inventory.

src/app/api/admin/articles/route.ts:11

Next -> #6 (p3)

#06 / Database Query / Implemented

bulk_orders

Code-backed collection bulk_orders; view reference fields in the model inventory.

src/app/api/v1/bulk-orders/route.ts:142

Next -> #7 (p3)

#07 / Response / Implemented

Stored domain response

Return persisted domain data to the service.

src/server/db.ts:100

END: no outgoing connection

#08 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #9 (p4)

Marketing & editorial data

Readable detail sheet 3/3. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Marketing & editorial data

Complete directed connection register / no omitted success, failure or proposed edges

#01 Domain persistence request -> #02 campaigns | Next

#02 campaigns -> #03 campaignEvents | Next

#03 campaignEvents -> #04 newsletter_subscribers | Next

#04 newsletter_subscribers -> #05 stories | Next

#05 stories -> #06 bulk_orders | Next

#06 bulk_orders -> #07 Stored domain response | Next

#01 Domain persistence request -> #08 Request failed | Failure | FAILURE

#08 Request failed -> #09 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: orders.campaignId -> campaigns.id (src/server/commerce.ts:178)

Note: campaignEvents.campaignId -> campaign identity (src/server/campaigns.ts:17)

MongoDB connection & Atlas boundary

Architecture overview. Readable node and connection sheets follow.



MongoDB connection & Atlas boundary

Readable detail sheet 1/2. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Database operation requested

Server code requests a shared MongoDB connection.

src/server/db.ts:30

Next -> #2 (p2)

#02 / IF / Switch / Implemented

MONGODB_URI supplied?

Absence rejects live operations; no credentials are displayed.

src/server/db.ts:31

Yes / accepted -> #3 (p2); No / rejected -> #7 (p3)

#03 / Database Query / Implemented

Shared native-driver pool

MongoClient.connect, maxPoolSize 10 and serverSelectionTimeoutMS 6000 are actual code settings.

src/server/db.ts:34

Next -> #4 (p2)

#04 / External API / Unverified

Hosted cluster / Atlas boundary

MongoDB hosting, Atlas cluster tier, region, replica placement and network access rules cannot be inferred from the connection code.

src/server/db.ts:33

Next -> #5 (p3)

MongoDB connection & Atlas boundary

Readable detail sheet 2/2. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

Database selection and transaction

MONGODB_DB override or ovia default; writes can share ClientSession.withTransaction.

src/server/db.ts:103

Next -> #6 (p3)

#06 / Response / Implemented

Repository result returned

Stored records or a surfaced database error return to business services.

src/server/db.ts:44

END: no outgoing connection

#07 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #8 (p3)

#08 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

MongoDB connection & Atlas boundary

Complete directed connection register / no omitted success, failure or proposed edges

#01 Database operation requested -> #02 MONGODB_URI supplied? | Next

#02 MONGODB_URI supplied? -> #03 Shared native-driver pool | Yes / accepted

#03 Shared native-driver pool -> #04 Hosted cluster / Atlas boundary | Next

#04 Hosted cluster / Atlas boundary -> #05 Database selection and transaction | Next

#05 Database selection and transaction -> #06 Repository result returned | Next

#02 MONGODB_URI supplied? -> #07 Request failed | No / rejected | FAILURE

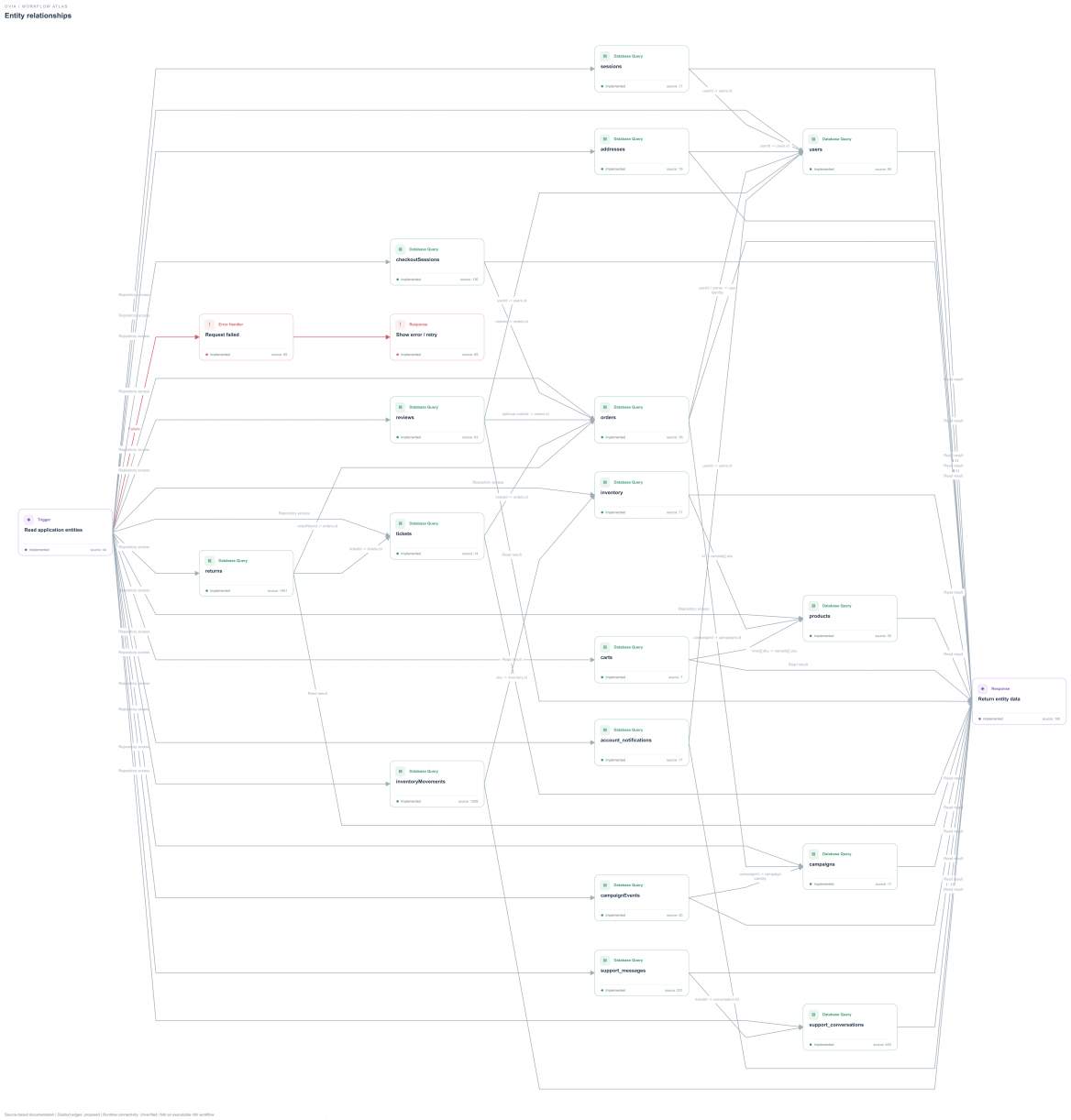
#07 Request failed -> #08 Show error / retry | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: The hosted-cluster node is a trust boundary, not proof of an Atlas deployment. No private network, replica topology or production configuration has been invented.

Entity relationships

Architecture overview. Readable node and connection sheets follow.



Entity relationships

Readable detail sheet 1/6. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Read application entities

Entity and reference view derived from actual collection access and reference fields.

src/server/db.ts:44

Failure -> #20 (p6); Repository access -> #2 (p2); Repository access -> #3 (p2); Repository access -> #4 (p2); Repository access -> #5 (p3); Repository access -> #6 (p3); Repository access -> #7 (p3); Repository access -> #8 (p3); Repository...

#02 / Database Query / Implemented

sessions

Application entity stored in sessions. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/app/api/admin/security/route.ts:17

Read result -> #19 (p6); userId -> users.id -> #3 (p2)

#03 / Database Query / Implemented

users

Application entity stored in users. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/app/api/admin/customers/route.ts:59

Read result -> #19 (p6)

#04 / Database Query / Implemented

addresses

Application entity stored in addresses. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/app/api/v1/account/addresses/[[...id]]/route.ts:19

Read result -> #19 (p6); userId -> users.id -> #3 (p2)

Entity relationships

Readable detail sheet 2/6. Node numbers and page references preserve every directed connection.

#05 / Database Query / Implemented

orders

Application entity stored in orders. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/app/api/admin/orders/[id]/document/route.ts:16

Read result -> #19 (p6); userId / owner -> user identity -> #3 (p2); campaignId -> campaigns.id -> #13 (p5)

#06 / Database Query / Implemented

checkoutSessions

Application entity stored in checkoutSessions. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/server/commerce.ts:136

Read result -> #19 (p6); orderId -> orders.id -> #5 (p3)

#07 / Database Query / Implemented

inventory

Application entity stored in inventory. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/app/api/v1/orders/[id]/return/route.ts:77

Read result -> #19 (p6); id -> variants[].sku -> #8 (p3)

#08 / Database Query / Implemented

products

Application entity stored in products. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/app/api/admin/product-taxonomy/route.ts:30

Read result -> #19 (p6)

Entity relationships

Readable detail sheet 3/6. Node numbers and page references preserve every directed connection.

#09 / Database Query / Implemented

carts

Application entity stored in carts. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/server/account-sign-in.ts:7

Read result -> #19 (p6); lines[].sku -> variants[].sku -> #8 (p3)

#10 / Database Query / Implemented

reviews

Application entity stored in reviews. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/app/api/v1/reviews/route.ts:63

Read result -> #19 (p6); userId -> users.id -> #3 (p2); optional orderId -> orders.id -> #5 (p3)

#11 / Database Query / Implemented

tickets

Application entity stored in tickets. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/app/api/v1/account/activity/route.ts:14

Read result -> #19 (p6); orderId -> orders.id -> #5 (p3)

#12 / Database Query / Implemented

returns

Application entity stored in returns. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/server/admin-service.ts:1401

Read result -> #19 (p6); ticketId -> tickets.id -> #11 (p4); orderRawId -> orders.id -> #5 (p3)

Entity relationships

Readable detail sheet 4/6. Node numbers and page references preserve every directed connection.

#13 / Database Query / Implemented

campaigns

Application entity stored in campaigns. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/server/campaigns.ts:11

Read result -> #19 (p6)

#14 / Database Query / Implemented

campaignEvents

Application entity stored in campaignEvents. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/server/campaigns.ts:20

Read result -> #19 (p6); campaignId -> campaign identity -> #13 (p5)

#15 / Database Query / Implemented

support_messages

Application entity stored in support_messages. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/app/api/v1/care/[[...path]]/route.ts:231

Read result -> #19 (p6); ticketId -> conversation ID -> #16 (p5)

#16 / Database Query / Implemented

support_conversations

Application entity stored in support_conversations. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/app/api/v1/care/[[...path]]/route.ts:453

Read result -> #19 (p6)

Entity relationships

Readable detail sheet 5/6. Node numbers and page references preserve every directed connection.

#17 / Database Query / Implemented

account_notifications

Application entity stored in account_notifications. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/app/api/v1/account/activity/route.ts:17

Read result -> #19 (p6); userId -> users.id -> #3 (p2)

#18 / Database Query / Implemented

inventoryMovements

Application entity stored in inventoryMovements. Relationship edges identify actual reference fields and imply no MongoDB foreign-key constraint.

src/server/admin-service.ts:1206

Read result -> #19 (p6); sku -> inventory.id -> #7 (p3)

#19 / Response / Implemented

Return entity data

Read results return to the application.

src/server/db.ts:100

END: no outgoing connection

#20 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #21 (p7)

Entity relationships

Readable detail sheet 6/6. Node numbers and page references preserve every directed connection.

#21 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

Entity relationships

Complete directed connection register / no omitted success, failure or proposed edges

#01 Read application entities -> #20 Request failed | Failure | FAILURE

#20 Request failed -> #21 Show error / retry | Next | FAILURE

#01 Read application entities -> #02 sessions | Repository access

#02 sessions -> #19 Return entity data | Read result

#01 Read application entities -> #03 users | Repository access

#03 users -> #19 Return entity data | Read result

#01 Read application entities -> #04 addresses | Repository access

#04 addresses -> #19 Return entity data | Read result

#01 Read application entities -> #05 orders | Repository access

#05 orders -> #19 Return entity data | Read result

#01 Read application entities -> #06 checkoutSessions | Repository access

#06 checkoutSessions -> #19 Return entity data | Read result

#01 Read application entities -> #07 inventory | Repository access

#07 inventory -> #19 Return entity data | Read result

#01 Read application entities -> #08 products | Repository access

#08 products -> #19 Return entity data | Read result

#01 Read application entities -> #09 carts | Repository access

#09 carts -> #19 Return entity data | Read result

#01 Read application entities -> #10 reviews | Repository access

#10 reviews -> #19 Return entity data | Read result

#01 Read application entities -> #11 tickets | Repository access

#11 tickets -> #19 Return entity data | Read result

Entity relationships

Directed connection register continued

#01 Read application entities -> #12 returns | Repository access

#12 returns -> #19 Return entity data | Read result

#01 Read application entities -> #13 campaigns | Repository access

#13 campaigns -> #19 Return entity data | Read result

#01 Read application entities -> #14 campaignEvents | Repository access

#14 campaignEvents -> #19 Return entity data | Read result

#01 Read application entities -> #15 support_messages | Repository access

#15 support_messages -> #19 Return entity data | Read result

#01 Read application entities -> #16 support_conversations | Repository access

#16 support_conversations -> #19 Return entity data | Read result

#01 Read application entities -> #17 account_notifications | Repository access

#17 account_notifications -> #19 Return entity data | Read result

#01 Read application entities -> #18 inventoryMovements | Repository access

#18 inventoryMovements -> #19 Return entity data | Read result

#02 sessions -> #03 users | userId -> users.id

#04 addresses -> #03 users | userId -> users.id

#05 orders -> #03 users | userId / owner -> user identity

#06 checkoutSessions -> #05 orders | orderId -> orders.id

#07 inventory -> #08 products | id -> variants[].sku

#09 carts -> #08 products | lines[].sku -> variants[].sku

#10 reviews -> #03 users | userId -> users.id

#10 reviews -> #05 orders | optional orderId -> orders.id

Entity relationships

Directed connection register continued

#11 tickets -> #05 orders | orderId -> orders.id

#12 returns -> #11 tickets | ticketId -> tickets.id

#12 returns -> #05 orders | orderRawId -> orders.id

#05 orders -> #13 campaigns | campaignId -> campaigns.id

#14 campaignEvents -> #13 campaigns | campaignId -> campaign identity

#15 support_messages -> #16 support_conversations | ticketId -> conversation ID

#17 account_notifications -> #03 users | userId -> users.id

#18 inventoryMovements -> #07 inventory | sku -> inventory.id

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: Reference edges represent application-level lookup relationships. Repository access/result edges preserve diagram entry and exit, not additional entity relationships.

Note: orders embeds paymentInfo, refunds[], shipping and logisticsOperations; users embeds accountWishlist. No nonexistent separate entity is introduced.